

Retrospective Evaluation of Soccer Players (REvSoc)

Seshadhri S

2025-07-18

2023-24 season

Goalkeeper Rating System: Methodology & Validation

In this analysis, I developed a multidimensional goalkeeper rating model designed to evaluate both conventional and modern aspects of goalkeeping performance. The model incorporates traditional shot-stopping metrics alongside advanced measures of distribution, sweeping actions, and cross management to reflect the evolving demands of the position.

Method Overview

The rating was divided into two core components:

Conventional Keeper Rating (CKR) — incorporating metrics like Saves, Goals Against (GA), Save Percentage, and Post-Shot Expected Goals (PSxG) overperformance.

Sweeper Keeper Rating (SKR) — capturing distribution accuracy, launch rates, defensive actions outside the box, average distance of defensive actions, and cross-stopping percentages.

Both components were normalized relative to their averages and combined to produce a Total Goalkeeper Rating.

Validation Process

To ensure the model was both credible and informative, I validated the Total Goalkeeper Rating against three independent metrics:

1. Goals Against (GA)
2. PSxG-GA per 90 (shot-stopping over expected)
3. Save Percentage

```
library(worldfootballR)
library(purrr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(Boruta)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats   1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v readr      2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(FactoMineR)
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(ggrepel)
```

Load in FBRef Data for the Big 5 Leagues

```
big5_player_standard <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "standard", team_or_player= "player")
big5_player_shooting <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "shooting", team_or_player= "player")
big5_player_passing <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "passing", team_or_player= "player")
big5_player_passing_types <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "passing_types", team_or_player= "player")
big5_player_gca <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "gca", team_or_player= "player")
big5_player_defense <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "defense", team_or_player= "player")
big5_player_possession <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "possession", team_or_player= "player")
big5_player_playing_time <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "playing_time", team_or_player= "player")
big5_player_misc <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "misc", team_or_player= "player")
big5_player_keepers <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "keepers", team_or_player= "player")
big5_player_keepers_adv <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "keepers_adv", team_or_player= "player")

prem_league_table <- fb_season_team_stats(country = "ENG", gender = "M", season_end_year = "2024", tier = "1")
seriea_league_table <- fb_season_team_stats(country = "ITA", gender = "M", season_end_year = "2024", tier = "1")
ligue1_league_table <- fb_season_team_stats(country = "FRA", gender = "M", season_end_year = "2024", tier = "1")
laliga_league_table <- fb_season_team_stats(country = "ESP", gender = "M", season_end_year = "2024", tier = "1")
bundesliga_league_table <- fb_season_team_stats(country = "GER", gender = "M", season_end_year = "2024", tier = "1")
```

Goalkeeper Ratings

```
##-----Goalkeeper-----
#>Have to divide this into Conventional Keeper Rating (CKR) and Sweeper Keeper Rating (SKR)
#>Take the keepers away from the playing times dataframe

gk_playing_time <- big5_player_playing_time %>% filter(Pos == "GK") %>% filter(MP_Playing.Time != 0)

df_list <- list(gk_playing_time, big5_player_keepers, big5_player_keepers_adv)
common_columns <- Reduce(intersect, lapply(df_list, colnames))

merged_df <- inner_join(big5_player_keepers, big5_player_keepers_adv, gk_playing_time, by = c("Season_E"))

gk_above_900 <- merged_df %>% filter(Min_Playing >= 900)

keepers_numeric <- gk_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
keepers_numeric <- keepers_numeric[, -1]
keepers_numeric <- na.omit(keepers_numeric)

# Scale the data (important for PCA since units differ)
keepers_scaled <- scale(keepers_numeric)
keepers_scaled <- keepers_scaled[, -1]
# Run PCA
keepers_pca <- PCA(keepers_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
keepers_pca$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  1.590736e+01          3.534968e+01          35.34968
## comp 2  7.042821e+00          1.565071e+01          51.00039
## comp 3  3.974753e+00          8.832785e+00          59.83318
## comp 4  3.249367e+00          7.220815e+00          67.05399
## comp 5  2.432045e+00          5.404545e+00          72.45854
## comp 6  1.959160e+00          4.353688e+00          76.81223
## comp 7  1.588120e+00          3.529156e+00          80.34138
## comp 8  1.371827e+00          3.048505e+00          83.38989
## comp 9  1.227811e+00          2.728469e+00          86.11836
## comp 10 9.881991e-01          2.195998e+00          88.31435
## comp 11 8.300425e-01          1.844539e+00          90.15889
## comp 12 7.347987e-01          1.632886e+00          91.79178
## comp 13 6.718116e-01          1.492915e+00          93.28469
## comp 14 5.623998e-01          1.249777e+00          94.53447
## comp 15 4.761981e-01          1.058218e+00          95.59269
## comp 16 3.815161e-01          8.478135e-01          96.44050
## comp 17 3.077148e-01          6.838106e-01          97.12431
## comp 18 2.327548e-01          5.172329e-01          97.64155
## comp 19 2.107799e-01          4.683997e-01          98.10994
## comp 20 1.666937e-01          3.704305e-01          98.48038
## comp 21 1.273148e-01          2.829217e-01          98.76330
## comp 22 1.235894e-01          2.746431e-01          99.03794
```

```
## comp 23 1.144442e-01      2.543204e-01      99.29226
## comp 24 8.306074e-02      1.845794e-01      99.47684
## comp 25 5.400544e-02      1.200121e-01      99.59685
## comp 26 5.213548e-02      1.158566e-01      99.71271
## comp 27 3.369793e-02      7.488429e-02      99.78759
## comp 28 2.803338e-02      6.229641e-02      99.84989
## comp 29 1.720388e-02      3.823084e-02      99.88812
## comp 30 1.440112e-02      3.200249e-02      99.92012
## comp 31 1.097971e-02      2.439936e-02      99.94452
## comp 32 9.211983e-03      2.047107e-02      99.96499
## comp 33 6.422978e-03      1.427328e-02      99.97927
## comp 34 4.191835e-03      9.315189e-03      99.98858
## comp 35 3.063175e-03      6.807056e-03      99.99539
## comp 36 1.800184e-03      4.000409e-03      99.99939
## comp 37 2.731768e-04      6.070594e-04      100.00000
## comp 38 1.698471e-06      3.774379e-06      100.00000
## comp 39 3.351979e-31      7.448841e-31      100.00000
## comp 40 1.303545e-31      2.896766e-31      100.00000
## comp 41 1.303545e-31      2.896766e-31      100.00000
## comp 42 1.303545e-31      2.896766e-31      100.00000
## comp 43 1.303545e-31      2.896766e-31      100.00000
## comp 44 1.303545e-31      2.896766e-31      100.00000
## comp 45 1.303545e-31      2.896766e-31      100.00000
```

```
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- keepers_pca$eig[1, "percentage of variance"]
dim2_variance <- keepers_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

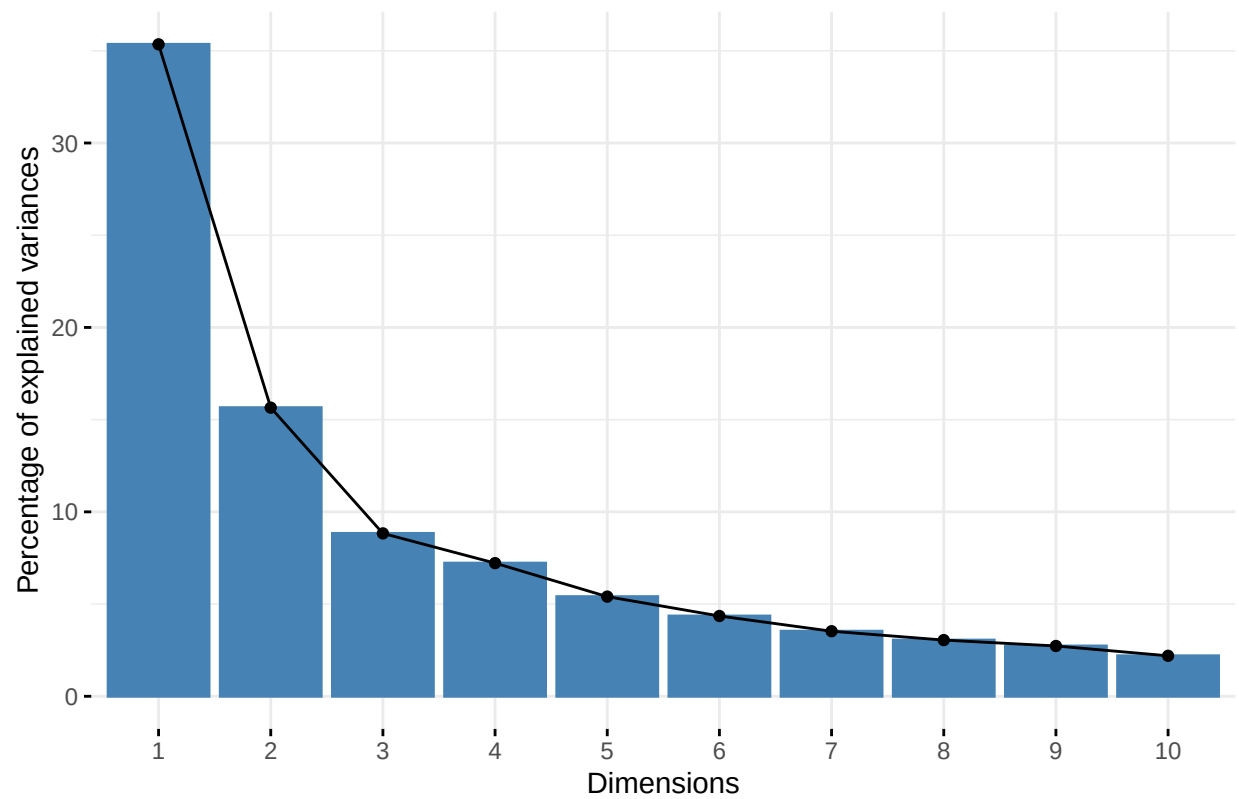
```
## [1] "Dim.1 weight: 0.693"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.307"
```

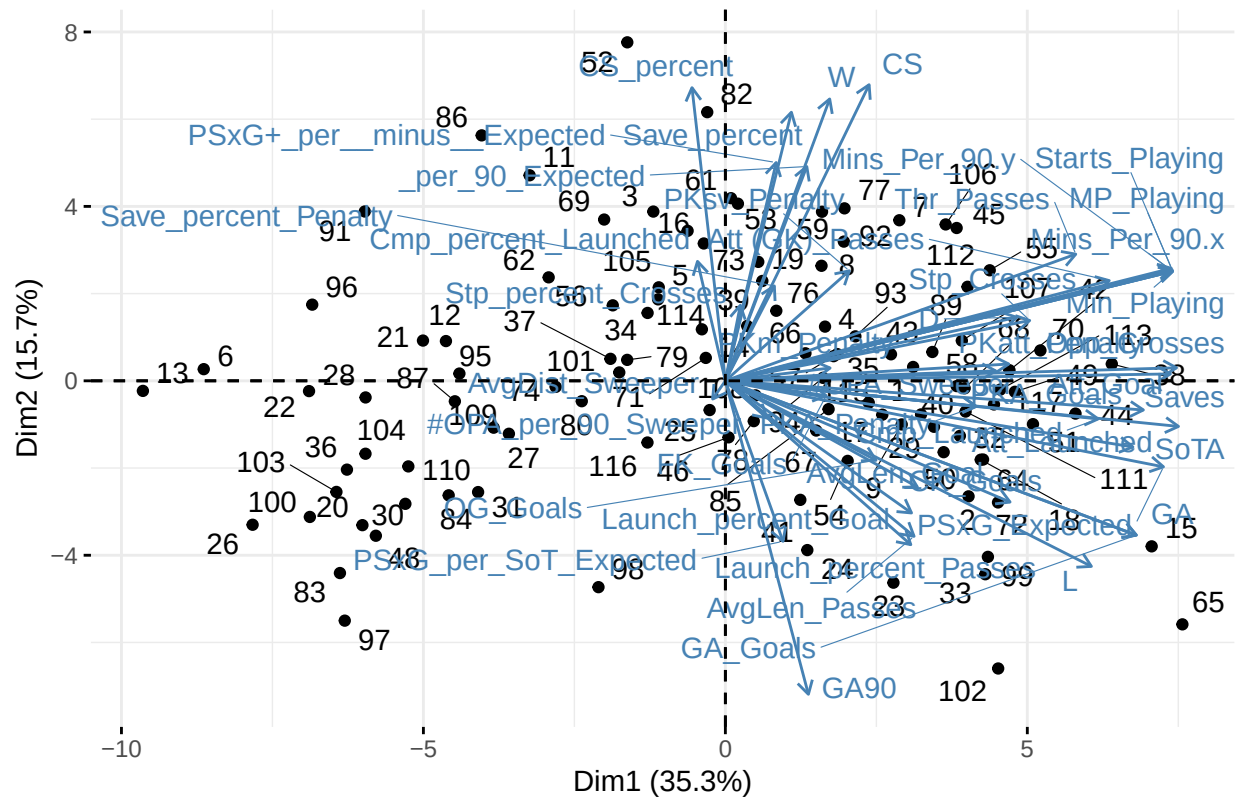
```
# Plot the variance explained
fviz_eig(keepers_pca)
```

Scree plot



```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(keepers_pca, repel = TRUE)
```

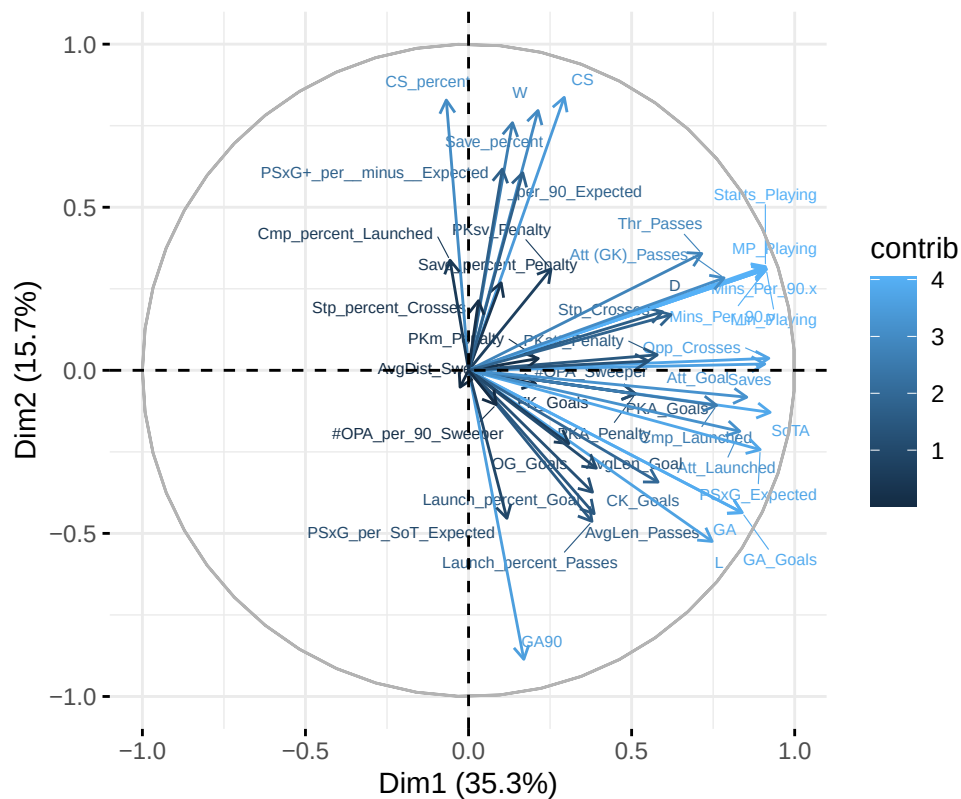
PCA – Biplot



Contributions of variables to PC1 and PC2

```
fviz_pca_var(keepers_pca,
  col.var = "contrib",
  labelsiz = 2, repel = TRUE)
```

Variables – PCA



```
variable_contrib <- data.frame(keepers_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

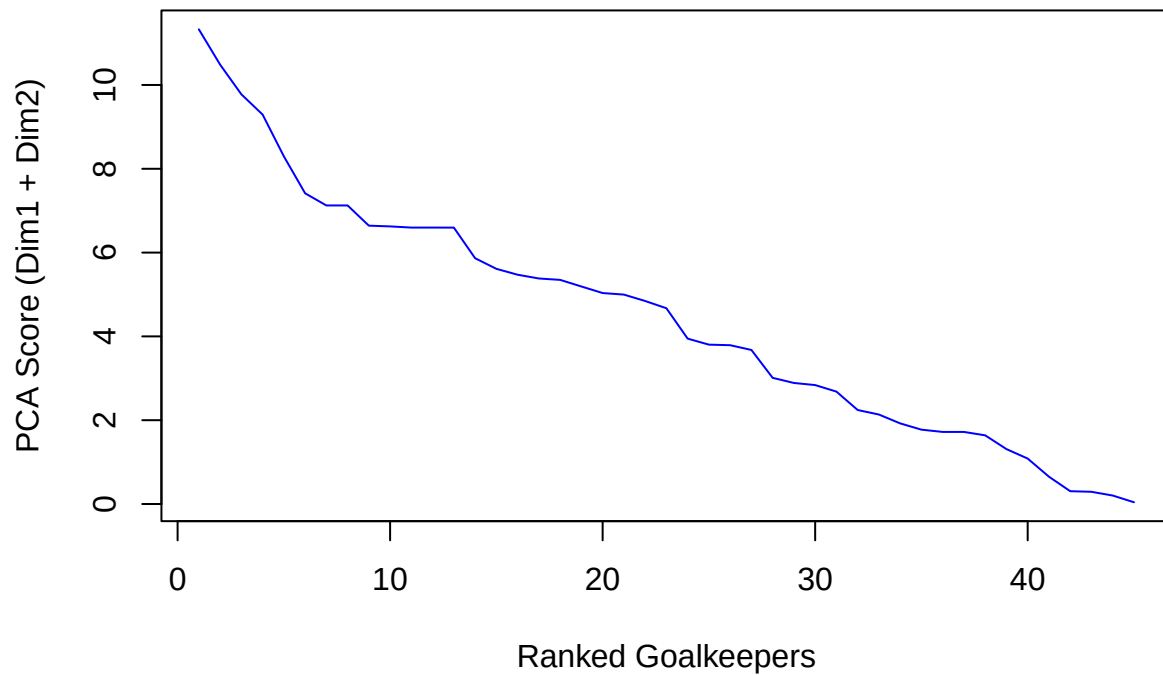
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 6.596986
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

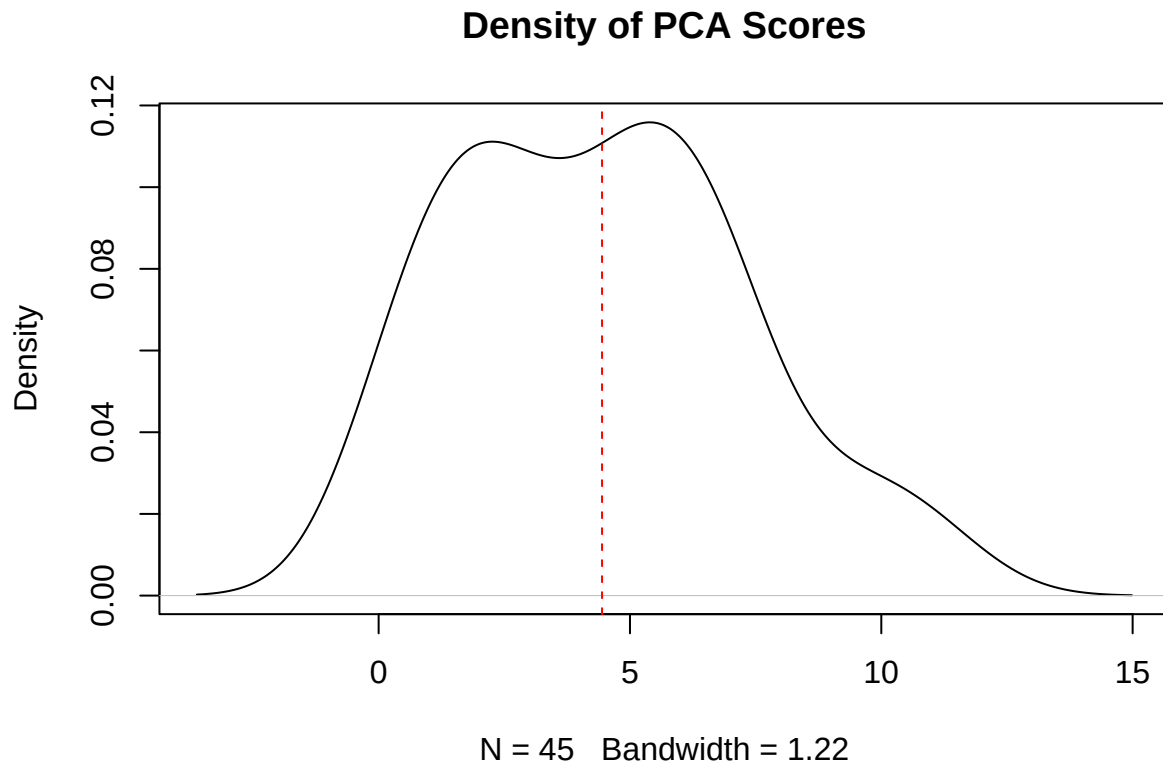
```
##      j1 j2  chi
## ESE  9 44 26.5
## EDE   7  1  NaN
```

```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 7.345754
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```



The PCA variable contribution plot shows a clear differentiation between shot stopping and sweeper keeper metrics

1. OPA_per_90_Sweeper and AvgDist_Sweeper are almost overlapping and long - These two are very closely related and both contribute heavily to PC1. TL;DR: your sweeper keeper stats move together and drive a big chunk of the differences between keepers.
2. PSxG+_per__minus__Expected (the expected goals prevented stat) points in a different direction and is quite long too - Strongly influential but not correlated with the sweeper actions. It's carrying weight independently, probably defining a classic shot-stopper profile.
3. Cmp_percent_Launched and Stp_percent_Crosses have shorter vectors, contributing less to the variance on these first two components. Could mean they're either:
 - Less variable between your keepers
 - Or important on other PCs (PC3, PC4...)

```
CKR <- with(gk_above_900,
  (Min_Playing *
    Saves * variable_contrib["Saves", "Weight"] *
    PSxG_Expected * variable_contrib["PSxG_Expected", "Weight"]) /
  (GA * variable_contrib["GA", "Weight"] * GA * SoTA * variable_contrib["SoTA", "Weight"])
)

SKR <- with(gk_above_900,
```

```

(
  (Cmp_Launched/Att_Launched) * variable_contrib["Cmp_Launched", "Weight"] +
  (Stp_Crosses/Opp_Crosses) * variable_contrib["Stp_Crosses", "Weight"] +
  (AvgLen_Passes/~ Att (GK)_Passes`) * variable_contrib["AvgLen_Passes", "Weight"] +
  (AvgDist_Sweeper/~ #OPA_Sweeper`) * variable_contrib["AvgDist_Sweeper", "Weight"] +
  (AvgLen_Goal/Att_Goal) * variable_contrib["AvgLen_Goal", "Weight"]
) / Min_Playing
)

avg_CKR <- mean(CKR, na.rm = TRUE)
avg_SKR <- mean(SKR, na.rm = TRUE)

CKR_Avg <- CKR / avg_CKR
SKR_Avg <- SKR / avg_SKR

Total_GK_Rating <- (CKR_Avg * dim1_weight) + (SKR_Avg * dim2_weight)

gk_ratings_final <- gk_above_900 %>%
  mutate(
    CKR_Avg = CKR_Avg,
    SKR_Avg = SKR_Avg,
    Total_GK_Rating = Total_GK_Rating
  ) %>%
  arrange(desc(Total_GK_Rating))

gk_ratings_24starts <- gk_ratings_final %>% filter(Starts_Playing >=25) %>% select(Squad, Comp, Player,
print(head(gk_ratings_24starts))

##           Squad      Comp           Player  CKR_Avg  SKR_Avg
## 1         Inter   Serie A      Yann Sommer 3.544517 0.6433091
## 2      Paris S-G   Ligue 1  Gianluigi Donnarumma 2.747923 0.8442182
## 3   Leverkusen Bundesliga  Lukáš Hrádecký 2.234062 0.6487758
## 4         Torino   Serie A  Vanja Milinković-Savić 1.804699 0.6015689
## 5          Monza   Serie A   Michele Di Gregorio 1.682678 0.6704946
## 6 Athletic Club   La Liga      Unai Simón 1.687612 0.5750991
##   Total_GK_Rating
## 1          2.654211
## 2          2.163725
## 3          1.747578
## 4          1.435489
## 5          1.372065
## 6          1.346211

#Correlation work between GA and GKR

cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$GA, use = "complete.obs", method = "spearman")

## [1] -0.8561138

cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$`PSxG+_per__minus__Expected`, use = "complete.obs")

## [1] 0.5790681

```

```
cor(gk_ratings_final$Total_GK_Rating, (gk_ratings_final$PSxG_Expected - gk_ratings_final$GA) / (gk_ratings_final$PSxG_Expected + gk_ratings_final$GA))
```

```
## [1] 0.549229
```

```
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$Save_percent, use = "complete.obs", method = "spearmanr")
```

```
## [1] 0.5933783
```

```
gk_ratings_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_GKR = mean(Total_GK_Rating))
print(gk_ratings_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp          Avg_GKR
##   <chr>         <dbl>
## 1 Bundesliga    0.897
## 2 La Liga       1.08
## 3 Ligue 1       1.05
## 4 Premier League 0.928
## 5 Serie A       1.04
```

```
gk_psxg_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_PsxG = mean(PSxG_Expected))
print(gk_psxg_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp          Avg_PsxG
##   <chr>         <dbl>
## 1 Bundesliga    42.7
## 2 La Liga       37.0
## 3 Ligue 1       38.1
## 4 Premier League 39.5
## 5 Serie A       32.6
```

```
cor(gk_ratings_league_avg$Avg_GKR, gk_psxg_league_avg$Avg_PsxG, method = "spearmanr")
```

```
## [1] -0.7
```

```
#Split them into Each League
```

```
pl_gk_ratings <- gk_ratings_final %>% filter(Comp == "Premier League")
bundesliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "Bundesliga")
seriea_gk_ratings <- gk_ratings_final %>% filter(Comp == "Serie A")
ligue1_gk_ratings <- gk_ratings_final %>% filter(Comp == "Ligue 1")
laliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "La Liga")
```

```
#Awards for each league
```

```
pl_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Premier League")
bundesliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Bundesliga")
seriea_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Serie A")
ligue1_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Ligue 1")
laliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "La Liga")
```

Interpretation

These correlations confirm that the model is strongly aligned with key outcome metrics while maintaining balance:

High correlation with Goals Against indicates outcome relevance.

Moderate correlations with PSxG-GA per 90 and Save Percentage confirm the model's shot-stopping validity without overfitting to a single dimension.

The moderate strength of these relationships reflects the rating's inclusion of modern, non-shot-stopping actions, enhancing its relevance to contemporary goalkeeper profiles.

Conclusion

This multidimensional rating offers a more comprehensive view of goalkeeper performance than conventional single-metric models. It captures the hybrid nature of the modern goalkeeper — one expected to contribute both in traditional shot-stopping and proactive defensive play — and stands up well to statistical validation against key performance outcomes.

Outfield Player Ratings

Defensive Player Ratings

```
#>DEFENSIVE PLAYER RATING
```

```
outfield_playing_time <- big5_player_playing_time %>% filter(Pos != "GK") %>% filter(Min_Playing.Time > 900)
outfield_df_list <- list(outfield_playing_time, big5_player_defense, big5_player_gca, big5_player_misc,
                        big5_player_passing, big5_player_passing_types, big5_player_possession,
                        big5_player_shooting, big5_player_standard)
common_columns_outfield <- Reduce(intersect, lapply(outfield_df_list, colnames))
print(common_columns_outfield)
```

```
## [1] "Season_End_Year" "Squad"          "Comp"          "Player"
## [5] "Nation"          "Pos"           "Age"           "Born"
## [9] "Url"
```

```
outfield_data <- outfield_df_list %>%
  reduce(left_join, by = c("Season_End_Year", "Squad", "Comp", "Player", "Nation", "Pos", "Age", "Born"))
outfield_data <- outfield_data %>%
  select(-Url)
```

```
outfield_defensive_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, Age, Born)
```

```
outfield_above_900 <- outfield_defensive_data %>% filter(Min_Playing.Time >= 900)
```

```
outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))
```

```
# Optional: remove NAs
```

```
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)
```

```

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[, -1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig

```

```

##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  6.65636603          44.3757736          44.37577
## comp 2  2.09470091          13.9646727          58.34045
## comp 3  1.68955456          11.2636970          69.60414
## comp 4  1.04866149           6.9910766          76.59522
## comp 5  0.82881649           5.5254433          82.12066
## comp 6  0.65678956           4.3785970          86.49926
## comp 7  0.53250718           3.5500479          90.04931
## comp 8  0.33824960           2.2549973          92.30431
## comp 9  0.29266578           1.9511052          94.25541
## comp 10 0.26148946           1.7432631          95.99867
## comp 11 0.20516189           1.3677459          97.36642
## comp 12 0.16774784           1.1183189          98.48474
## comp 13 0.13809158           0.9206106          99.40535
## comp 14 0.06112502           0.4075001          99.81285
## comp 15 0.02807262           0.1871508         100.00000

```

```

# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))

```

```
## [1] "Dim.1 weight: 0.761"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

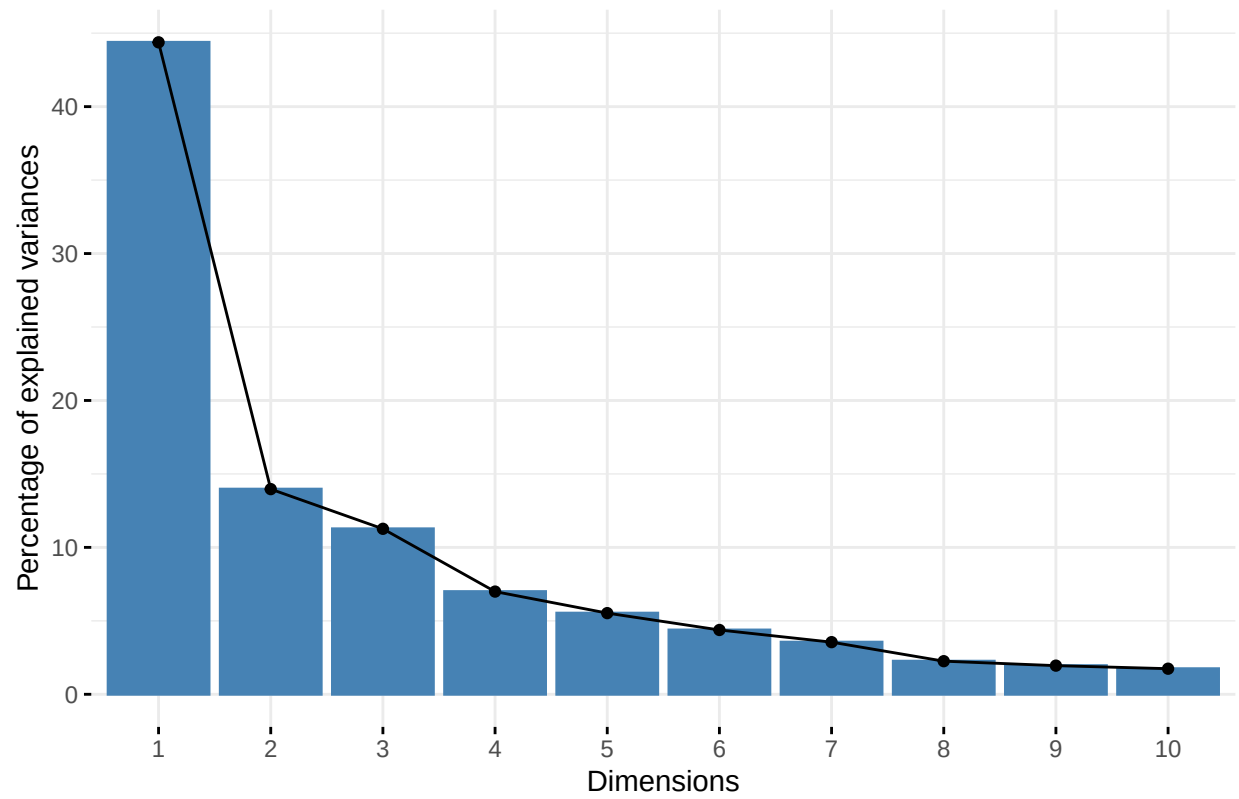
```
## [1] "Dim.2 weight: 0.239"
```

```

# Plot the variance explained
fviz_eig(outfield_pca)

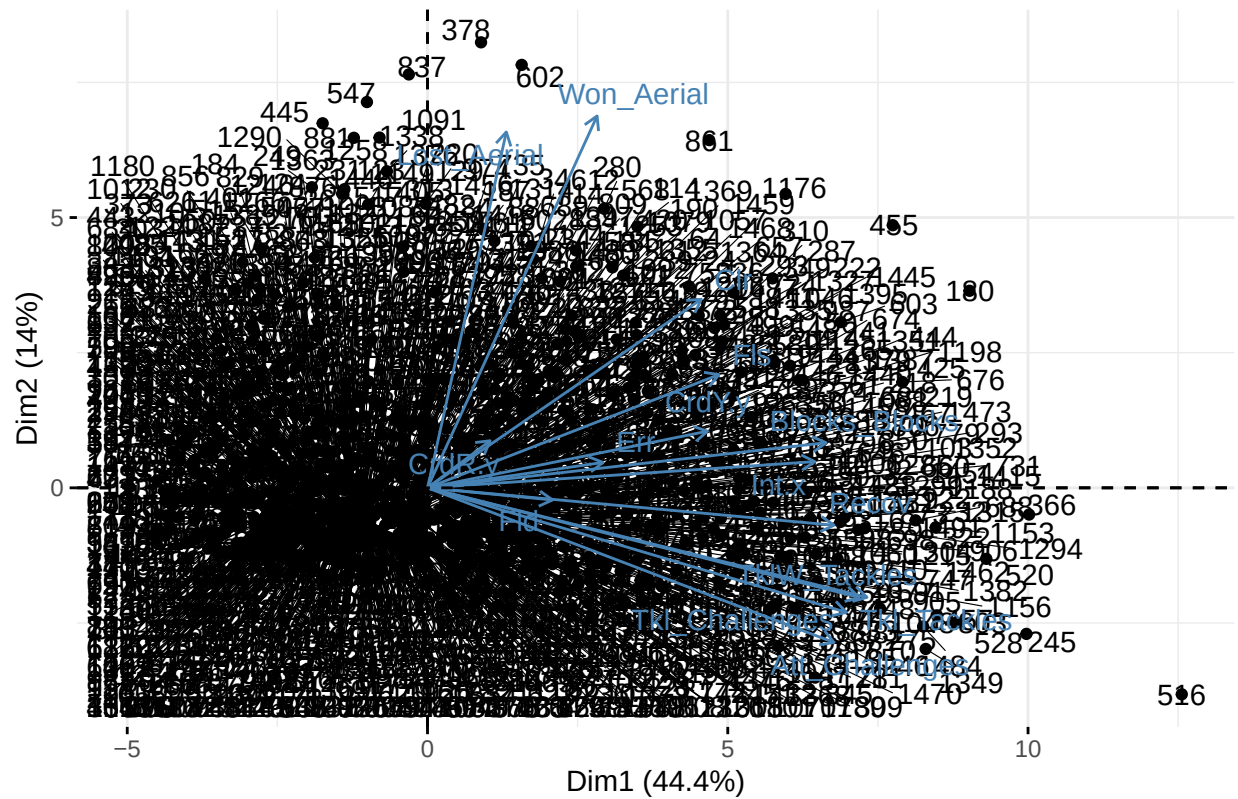
```

Scree plot

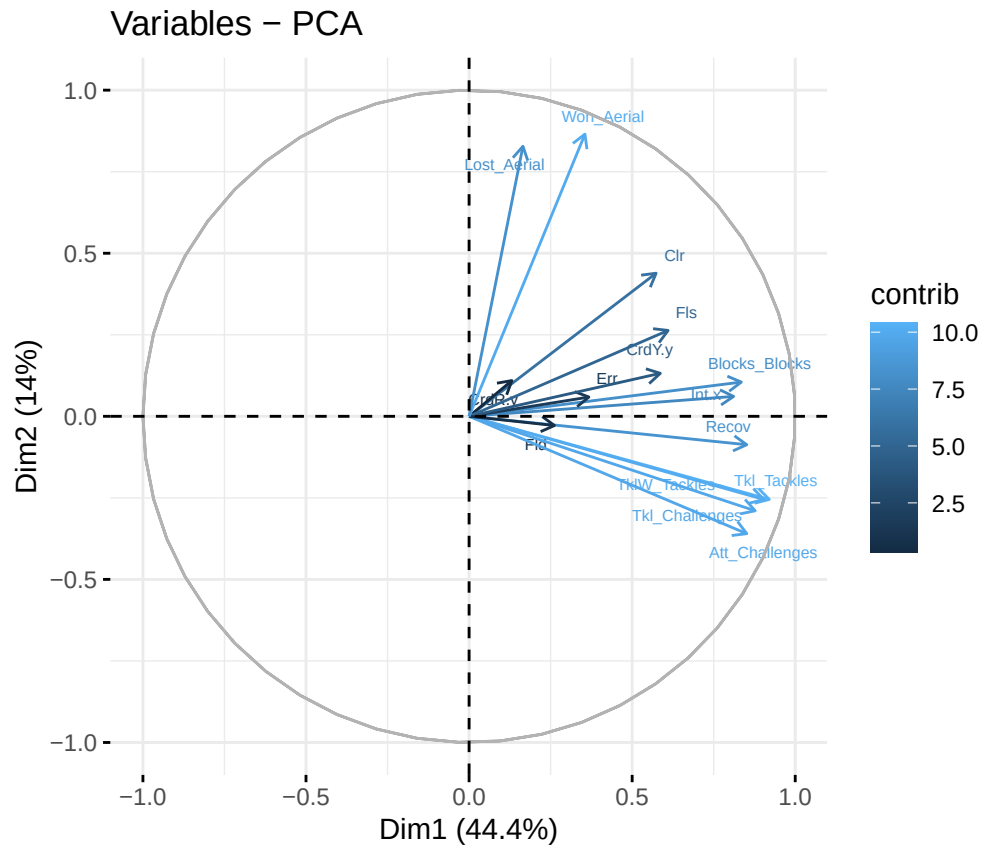


```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```

PCA – Biplot



```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
  col.var = "contrib",
  labelsize = 2, repel = TRUE)
```



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

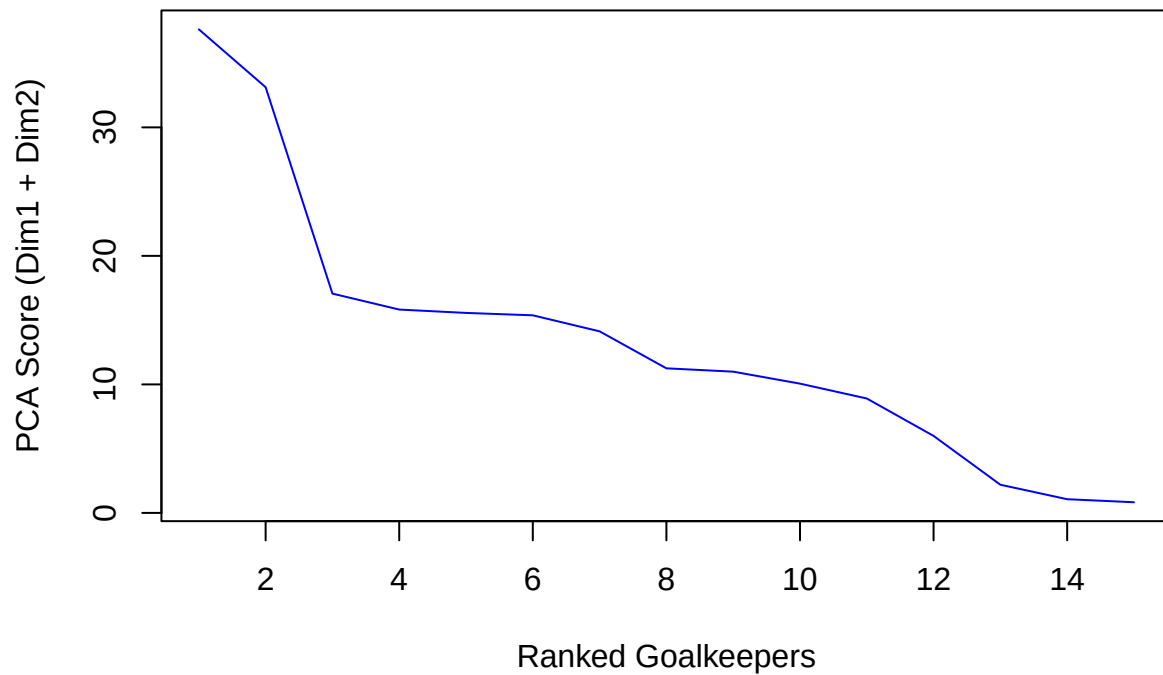
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 15.69353
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
      ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2 chi
## ESE  3 14 8.5
## EDE  3  1 NaN
```

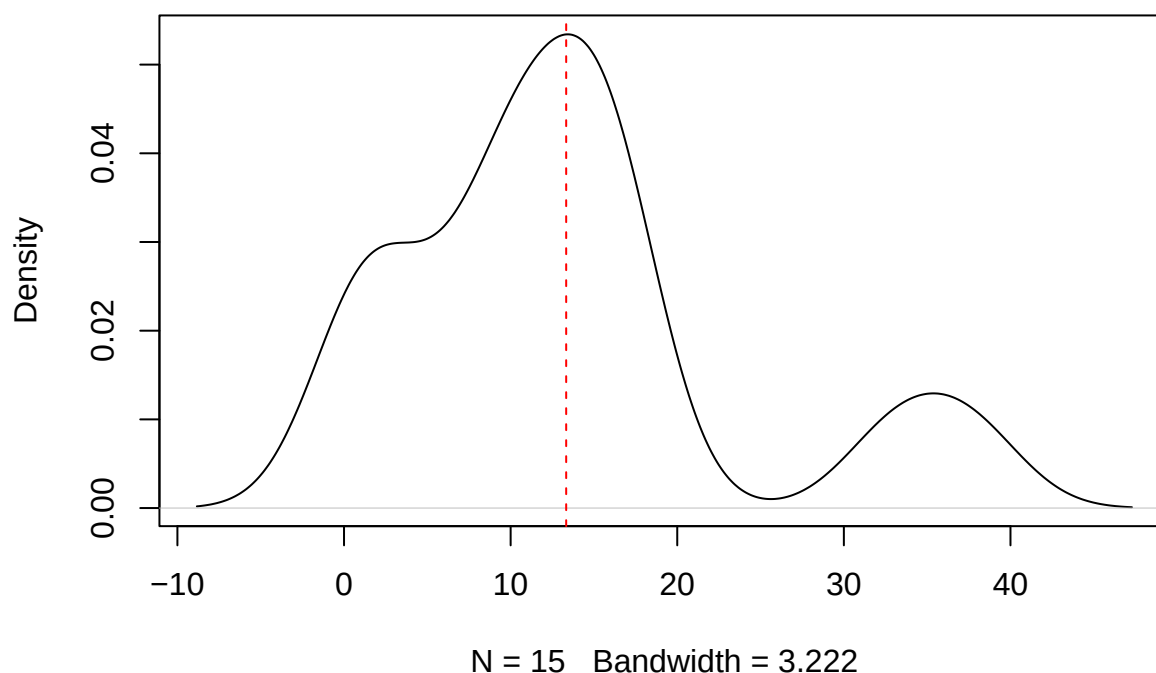
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 23.80979
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

Density of PCA Scores



#####

```
outfield_dpr <- outfield_data %>% mutate(Defensive_Player_Rtg = ((Tkl_Tackles* variable_contrib["Tkl_Tackles"])/100))
```

```
outfield_dpr_filtered <- outfield_dpr %>%
  filter(is.finite(Defensive_Player_Rtg), !is.na(Defensive_Player_Rtg))
```

```
#> Get the Average possession for every team in the top 5 leagues (will be useful for other metrics too.)
#> For defenders, divide DPR by (100 - Avg. Poss.)
#> First things first, let's extract the possession data at team level
```

```
big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "possession", team_level= TRUE)
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 192
## Columns: 30
## $ Season_End_Year      <int> 2024, 2024, 2024, 2024, 2024, 2024, 2024, 2024, 20~
## $ Squad                <chr> "Alavés", "Alavés", "Almería", "Almería", "Arsenal~
## $ Comp                 <chr> "La Liga", "La Liga", "La Liga", "La Liga", "Premi~
## $ Team_or_Opponent     <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players          <dbl> 30, 30, 35, 35, 25, 25, 31, 31, 29, 29, 27, 27, 27~
## $ Poss                 <dbl> 41.8, 58.2, 46.4, 53.6, 58.2, 41.8, 53.1, 46.9, 51~
## $ Mins_Per_90          <dbl> 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38~
```

```
## $ Touches_Touches      <dbl> 18179, 24410, 20902, 23390, 26493, 20293, 23059, 2~
## $ 'Def Pen_Touches'    <dbl> 2162, 2398, 2654, 2099, 1974, 3159, 2807, 2242, 22~
## $ 'Def 3rd_Touches'    <dbl> 6122, 7446, 7711, 6542, 6714, 8333, 8171, 6844, 74~
## $ 'Mid 3rd_Touches'    <dbl> 7564, 11388, 9323, 10880, 11407, 8198, 9636, 9746, ~
## $ 'Att 3rd_Touches'    <dbl> 4706, 5787, 4094, 6169, 8560, 3972, 5459, 4982, 61~
## $ 'Att Pen_Touches'    <dbl> 764, 818, 614, 1003, 1480, 570, 1010, 893, 912, 61~
## $ Live_Touches         <dbl> 18172, 24402, 20900, 23381, 26483, 20290, 23055, 2~
## $ Att_Take              <dbl> 625, 698, 667, 641, 671, 612, 757, 624, 618, 667, ~
## $ Succ_Take             <dbl> 310, 339, 299, 302, 283, 272, 345, 290, 286, 306, ~
## $ Succ_percent_Take     <dbl> 49.6, 48.6, 44.8, 47.1, 42.2, 44.4, 45.6, 46.5, 46~
## $ Tkld_Take             <dbl> 254, 290, 314, 280, 304, 271, 315, 260, 270, 291, ~
## $ Tkld_percent_Take     <dbl> 40.6, 41.5, 47.1, 43.7, 45.3, 44.3, 41.6, 41.7, 43~
## $ Carries_Carries      <dbl> 8574, 13762, 11128, 13182, 16297, 11433, 14451, 12~
## $ TotDist_Carries       <dbl> 48049, 75816, 59196, 69802, 81171, 59629, 78629, 6~
## $ PrgDist_Carries       <dbl> 25494, 41243, 30444, 36749, 41510, 28836, 41239, 3~
## $ PrgC_Carries          <dbl> 550, 744, 590, 751, 821, 534, 775, 640, 715, 559, ~
## $ Final_Third_Carries   <dbl> 364, 542, 424, 563, 575, 409, 490, 423, 519, 407, ~
## $ CPA_Carries           <dbl> 138, 187, 132, 204, 327, 159, 217, 225, 175, 129, ~
## $ Mis_Carries           <dbl> 572, 663, 550, 518, 525, 455, 477, 462, 621, 634, ~
## $ Dis_Carries           <dbl> 299, 288, 269, 353, 375, 339, 416, 287, 377, 322, ~
## $ Rec_Receiving         <dbl> 9972, 16187, 12660, 15316, 18917, 12601, 15824, 14~
## $ PrgR_Receiving        <dbl> 1252, 1487, 1082, 1620, 2088, 922, 1438, 1232, 173~
## $ Url                   <chr> "https://fbref.com/en/squads/8d6fd021/2023-2024/A1~
```

```
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% se
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_dpr_filtered_possopp <- outfield_dpr_filtered %>% left_join(big5_team_possession_opponent, by =
outfield_dpr_filtered_possopp_possteam <- outfield_dpr_filtered_possopp %>% left_join(big5_team_possess

outfield_DPR <- outfield_dpr_filtered_possopp_possteam %>% mutate(Poss.Adj.DPR = Defensive_Player_Rtg*1
avg_dpr <- mean(outfield_DPR$Poss.Adj.DPR)
outfield_DPR <- outfield_DPR %>% mutate(Poss.Adj.DPR = (Poss.Adj.DPR/avg_dpr)) %>% arrange(desc(Poss.Ad

pl_dpr <- outfield_DPR %>% filter(Comp == "Premier League")
ligue1_dpr <- outfield_DPR %>% filter(Comp == "Ligue 1")
seriea_dpr <- outfield_DPR %>% filter(Comp == "Serie A")
bundesliga_dpr <- outfield_DPR %>% filter(Comp == "Bundesliga")
laliga_dpr <- outfield_DPR %>% filter(Comp == "La Liga")
```

#> Outfield_DPR is what you have to carry forward because you cannot have players ineligible for any ca

Offensive Creation Rating

```
#>OFFENSIVE CREATION RATING
```

```
outfield_ocr_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, Age, M

colnames(outfield_ocr_data)[21] <- "Att_3rd_Touches"
colnames(outfield_ocr_data)[22] <- "Att_Pen_Touches"
```

```

outfield_above_900 <- outfield_ocr_data%>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)

outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[, -1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig

```

```

##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  13.045662879          46.591653139          46.59165
## comp 2   7.868706198          28.102522135          74.69418
## comp 3   1.946131246           6.950468735          81.64464
## comp 4   1.141379296           4.076354627          85.72100
## comp 5   0.869515258           3.105411636          88.82641
## comp 6   0.617895233           2.206768690          91.03318
## comp 7   0.523816130           1.870771894          92.90395
## comp 8   0.366228181           1.307957789          94.21191
## comp 9   0.288378072           1.029921685          95.24183
## comp 10  0.241259253           0.861640189          96.10347
## comp 11  0.165973747           0.592763383          96.69623
## comp 12  0.147278547           0.525994812          97.22223
## comp 13  0.128847028           0.460167957          97.68240
## comp 14  0.112261684           0.400934587          98.08333
## comp 15  0.092266341           0.329522646          98.41285
## comp 16  0.081732921           0.291903288          98.70476
## comp 17  0.075862194           0.270936407          98.97569
## comp 18  0.068792804           0.245688585          99.22138
## comp 19  0.055646124           0.198736158          99.42012
## comp 20  0.041515898           0.148271063          99.56839
## comp 21  0.025002612           0.089295042          99.65768
## comp 22  0.023049248           0.082318744          99.74000
## comp 23  0.021785013           0.077803619          99.81781
## comp 24  0.019376691           0.069202467          99.88701
## comp 25  0.013208192           0.047172116          99.93418
## comp 26  0.011835513           0.042269689          99.97645
## comp 27  0.005390760           0.019252716          99.99570
## comp 28  0.001202937           0.004296202         100.00000

```

```

# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)

```

```
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

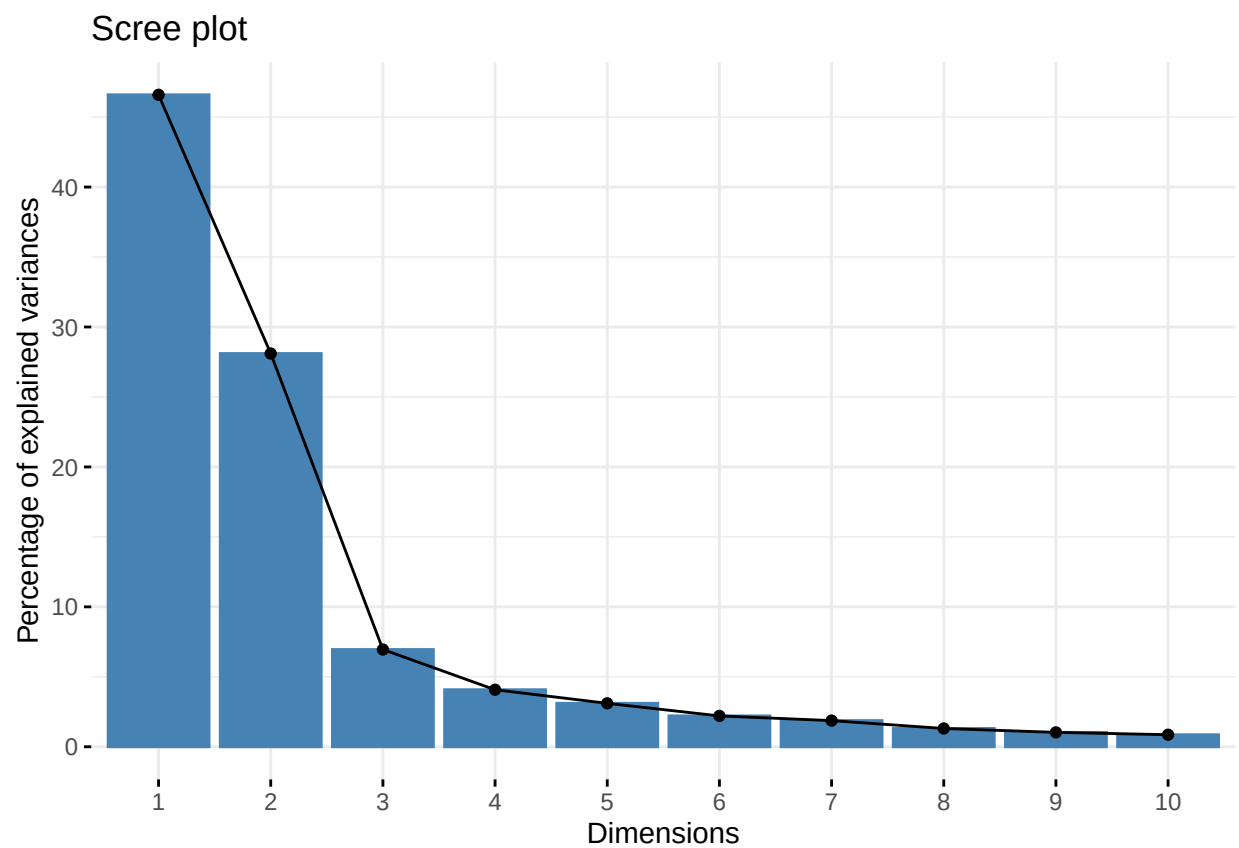
print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.624"
```

```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

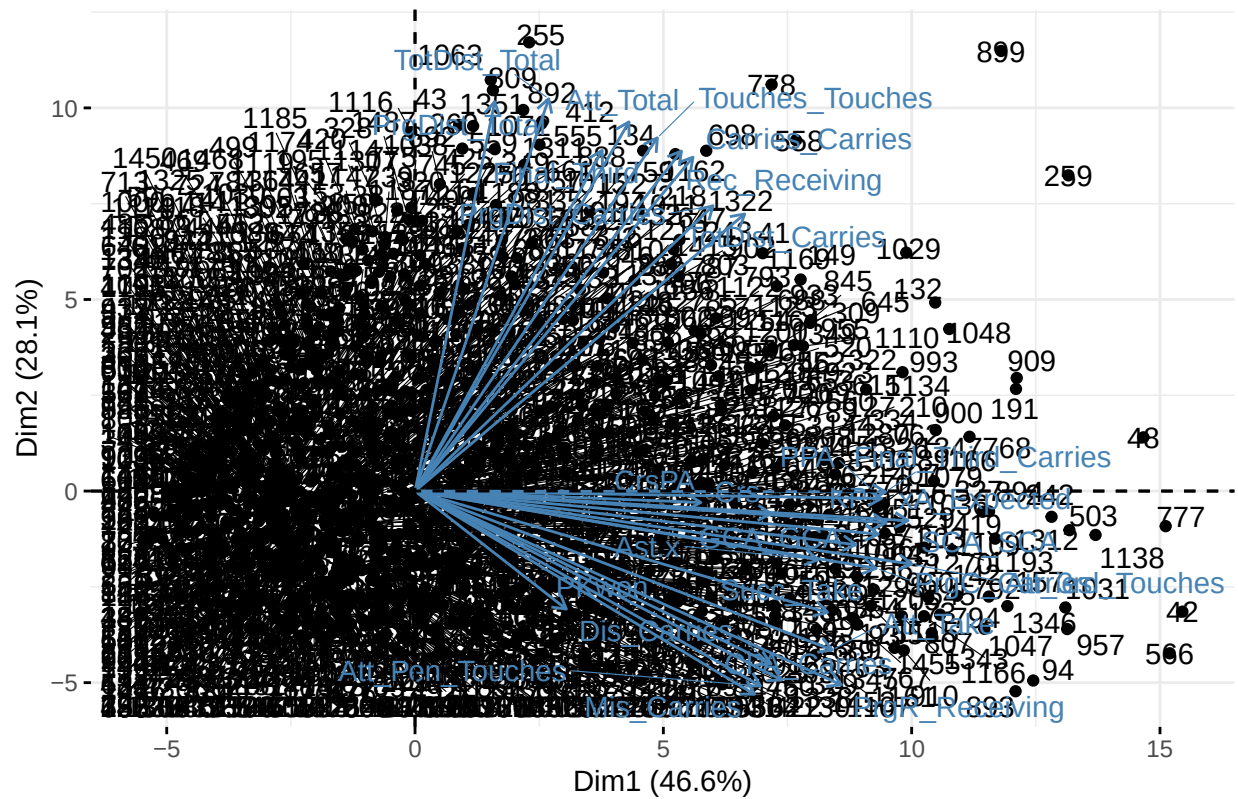
```
## [1] "Dim.2 weight: 0.376"
```

```
# Plot the variance explained
fviz_eig(outfield_pca)
```



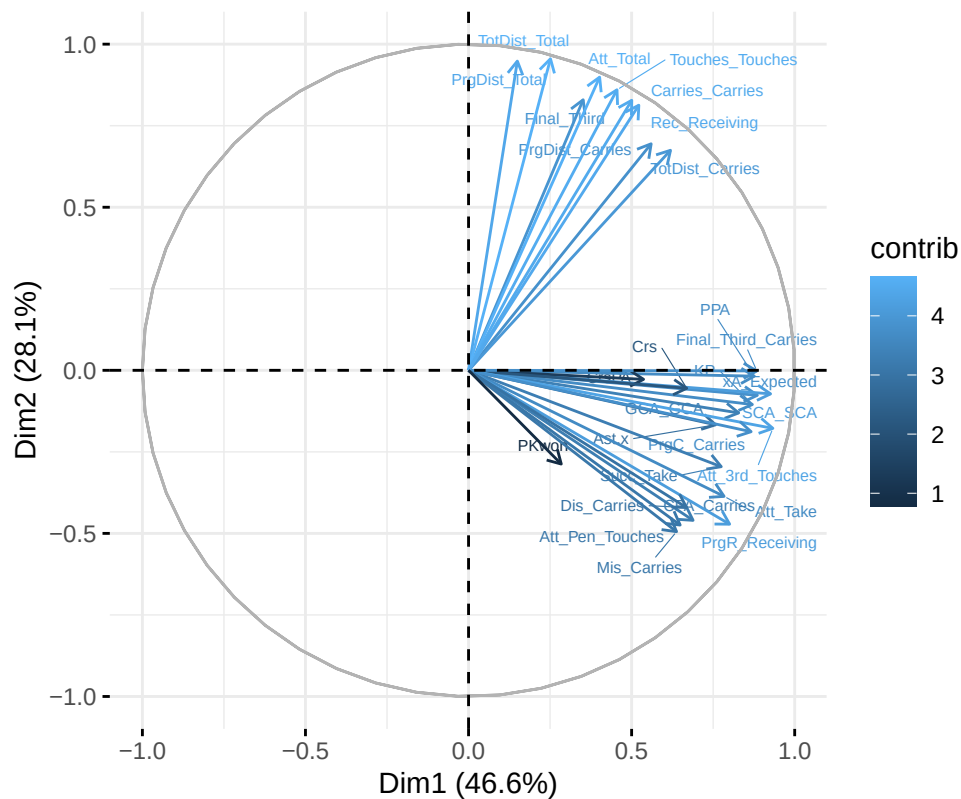
```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```

PCA – Biplot



```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
  col.var = "contrib",
  labelsize = 2, repel = TRUE)
```

Variables – PCA



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

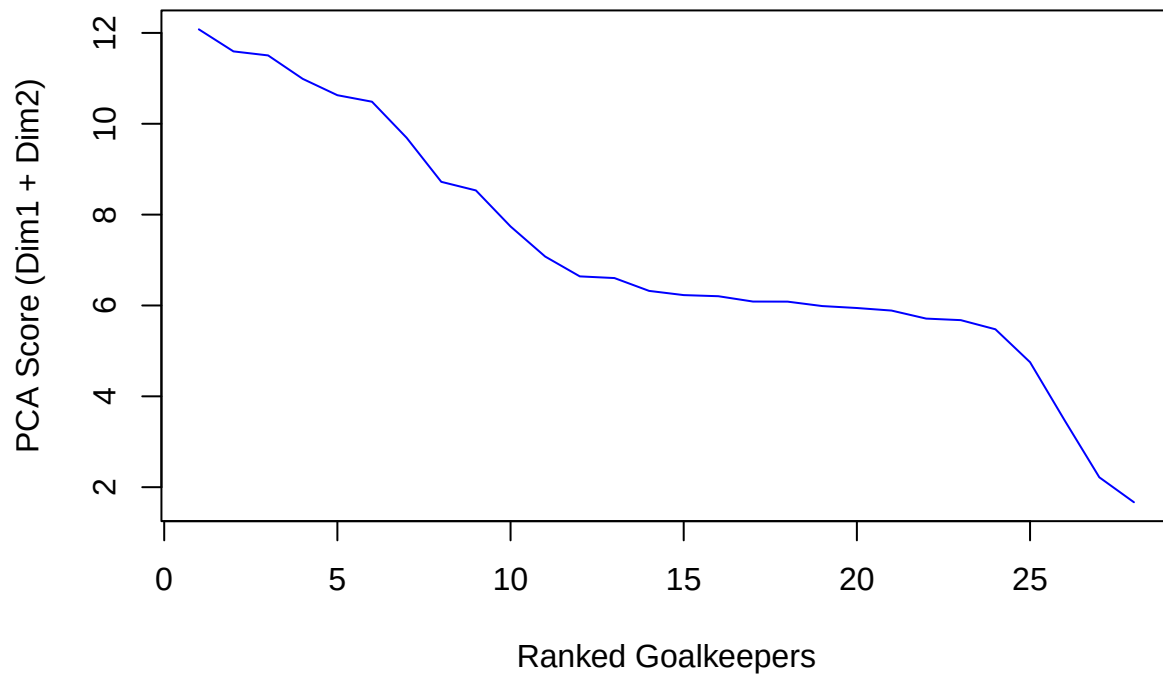
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 8.964518
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
      ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2 chi
## ESE 12 24  18
## EDE 12 24  18
```

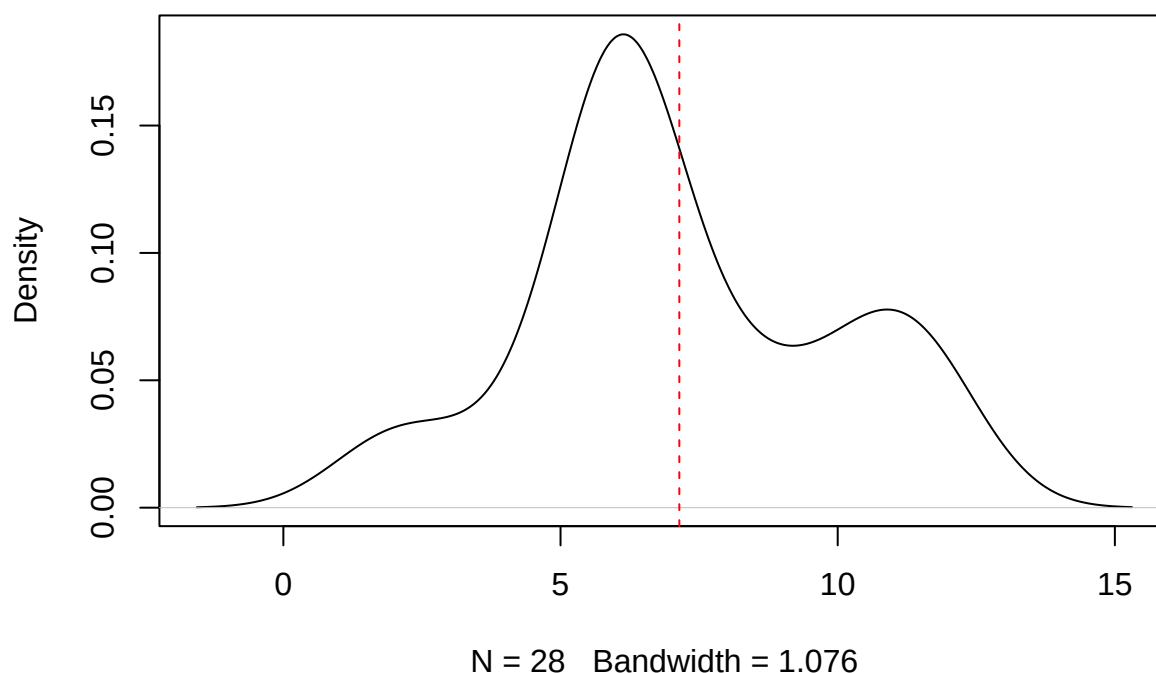
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 9.88169
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

Density of PCA Scores



```
#####
```

```
#> OFFENSIVE CREATION RATING
```

```
outfield_DPR_ocr <- outfield_above_900 %>% mutate(Off_Creation_Rtg = (((Ast.x *variable_contrib["Ast.x"]
setdiff(c("Ast.x", "xA_Expected", "SCA_SCA", "GCA_GCA", "KP", "Final_Third", "PPA", "Att_Total", "CrsPA
```

```
## character(0)
```

```
big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2024, stat_type= "possession", t
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 192
## Columns: 30
## $ Season_End_Year    <int> 2024, 2024, 2024, 2024, 2024, 2024, 2024, 2024, 20~
## $ Squad              <chr> "Alavés", "Alavés", "Almería", "Almería", "Arsenal~
## $ Comp               <chr> "La Liga", "La Liga", "La Liga", "La Liga", "Premi~
## $ Team_or_Opponent   <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players        <dbl> 30, 30, 35, 35, 25, 25, 31, 31, 29, 29, 27, 27, 27~
## $ Poss               <dbl> 41.8, 58.2, 46.4, 53.6, 58.2, 41.8, 53.1, 46.9, 51~
## $ Mins_Per_90        <dbl> 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38~
## $ Touches_Touches    <dbl> 18179, 24410, 20902, 23390, 26493, 20293, 23059, 2~
## $ 'Def Pen_Touches'  <dbl> 2162, 2398, 2654, 2099, 1974, 3159, 2807, 2242, 22~
```

```
## $ 'Def 3rd_Touches' <dbl> 6122, 7446, 7711, 6542, 6714, 8333, 8171, 6844, 74~
## $ 'Mid 3rd_Touches' <dbl> 7564, 11388, 9323, 10880, 11407, 8198, 9636, 9746, ~
## $ 'Att 3rd_Touches' <dbl> 4706, 5787, 4094, 6169, 8560, 3972, 5459, 4982, 61~
## $ 'Att Pen_Touches' <dbl> 764, 818, 614, 1003, 1480, 570, 1010, 893, 912, 61~
## $ Live_Touches <dbl> 18172, 24402, 20900, 23381, 26483, 20290, 23055, 2~
## $ Att_Take <dbl> 625, 698, 667, 641, 671, 612, 757, 624, 618, 667, ~
## $ Succ_Take <dbl> 310, 339, 299, 302, 283, 272, 345, 290, 286, 306, ~
## $ Succ_percent_Take <dbl> 49.6, 48.6, 44.8, 47.1, 42.2, 44.4, 45.6, 46.5, 46~
## $ Tkld_Take <dbl> 254, 290, 314, 280, 304, 271, 315, 260, 270, 291, ~
## $ Tkld_percent_Take <dbl> 40.6, 41.5, 47.1, 43.7, 45.3, 44.3, 41.6, 41.7, 43~
## $ Carries_Carries <dbl> 8574, 13762, 11128, 13182, 16297, 11433, 14451, 12~
## $ TotDist_Carries <dbl> 48049, 75816, 59196, 69802, 81171, 59629, 78629, 6~
## $ PrgDist_Carries <dbl> 25494, 41243, 30444, 36749, 41510, 28836, 41239, 3~
## $ PrgC_Carries <dbl> 550, 744, 590, 751, 821, 534, 775, 640, 715, 559, ~
## $ Final_Third_Carries <dbl> 364, 542, 424, 563, 575, 409, 490, 423, 519, 407, ~
## $ CPA_Carries <dbl> 138, 187, 132, 204, 327, 159, 217, 225, 175, 129, ~
## $ Mis_Carries <dbl> 572, 663, 550, 518, 525, 455, 477, 462, 621, 634, ~
## $ Dis_Carries <dbl> 299, 288, 269, 353, 375, 339, 416, 287, 377, 322, ~
## $ Rec_Receiving <dbl> 9972, 16187, 12660, 15316, 18917, 12601, 15824, 14~
## $ PrgR_Receiving <dbl> 1252, 1487, 1082, 1620, 2088, 922, 1438, 1232, 173~
## $ Url <chr> "https://fbref.com/en/squads/8d6fd021/2023-2024/A1~
```

```
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% se
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_opponent, by = "Squad", relation
outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_team, by = "Squad", relationship

outfield_DPR_ocr_possadj <- outfield_DPR_ocr %>% mutate(Poss.adj.OCR = (Off_Creation_Rtg/Poss_Team)*100)
outfield_DPR_ocr_possadj <- outfield_DPR_ocr_possadj %>% filter(Poss.adj.OCR != "NaN")
avg_ocr <- mean(outfield_DPR_ocr_possadj$Poss.adj.OCR)
outfield_DPR_OCR <- outfield_DPR_ocr_possadj %>% mutate(Poss.adj.OCR = Poss.adj.OCR/avg_ocr) %>% arrang

outfield_DPR_OCR_5ast <- outfield_DPR_OCR %>% filter(Ast.x >= 5)
```

Goalscorer Rating

#>GOALSCORER RATING

```
outfield_gsr_data <- outfield_data %>%
  mutate(
    Goals_per_xG = ifelse(xG_Expected.x > 0, Gls_Standard / xG_Expected.x, 0),
    G_per_Shot = ifelse(Sh_Standard > 0, Gls_Standard / Sh_Standard, 0),
    xG_per_Shot = ifelse(Sh_Standard > 0, xG_Expected.x / Sh_Standard, 0))

outfield_gsr_data <- outfield_gsr_data %>% mutate(Gls_per_xG = Gls/xG_Expected.x) %>% select(Season_End,

colnames(outfield_gsr_data)[10] <- "xG_Expected"

outfield_above_900 <- outfield_gsr_data %>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)
```

```
outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))
```

```
# Optional: remove NAs
```

```
outfield_numeric <- outfield_numeric[, -1]
```

```
outfield_numeric <- na.omit(outfield_numeric)
```

```
# Scale the data (important for PCA since units differ)
```

```
outfield_scaled <- scale(outfield_numeric)
```

```
outfield_scaled <- outfield_scaled[, -1]
```

```
# Run PCA
```

```
outfield_pca <- PCA(outfield_scaled, graph = FALSE)
```

```
# View eigenvalues (variance explained by each component)
```

```
outfield_pca$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  5.832315e+00          5.302105e+01          53.02105
## comp 2  2.424451e+00          2.204047e+01          75.06152
## comp 3  1.140510e+00          1.036827e+01          85.42979
## comp 4  6.794771e-01          6.177064e+00          91.60685
## comp 5  4.491585e-01          4.083259e+00          95.69011
## comp 6  1.925224e-01          1.750203e+00          97.44031
## comp 7  1.504787e-01          1.367988e+00          98.80830
## comp 8  8.075599e-02          7.341454e-01          99.54245
## comp 9  4.550957e-02          4.137233e-01          99.95617
## comp 10 4.821319e-03          4.383017e-02          100.00000
## comp 11 2.274417e-31          2.067652e-30          100.00000
```

```
# Extract variance explained by Dim.1 and Dim.2
```

```
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
```

```
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]
```

```
# Normalize so they sum to 1 (as relative weights)
```

```
total_variance <- dim1_variance + dim2_variance
```

```
dim1_weight <- dim1_variance / total_variance
```

```
dim2_weight <- dim2_variance / total_variance
```

```
print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.706"
```

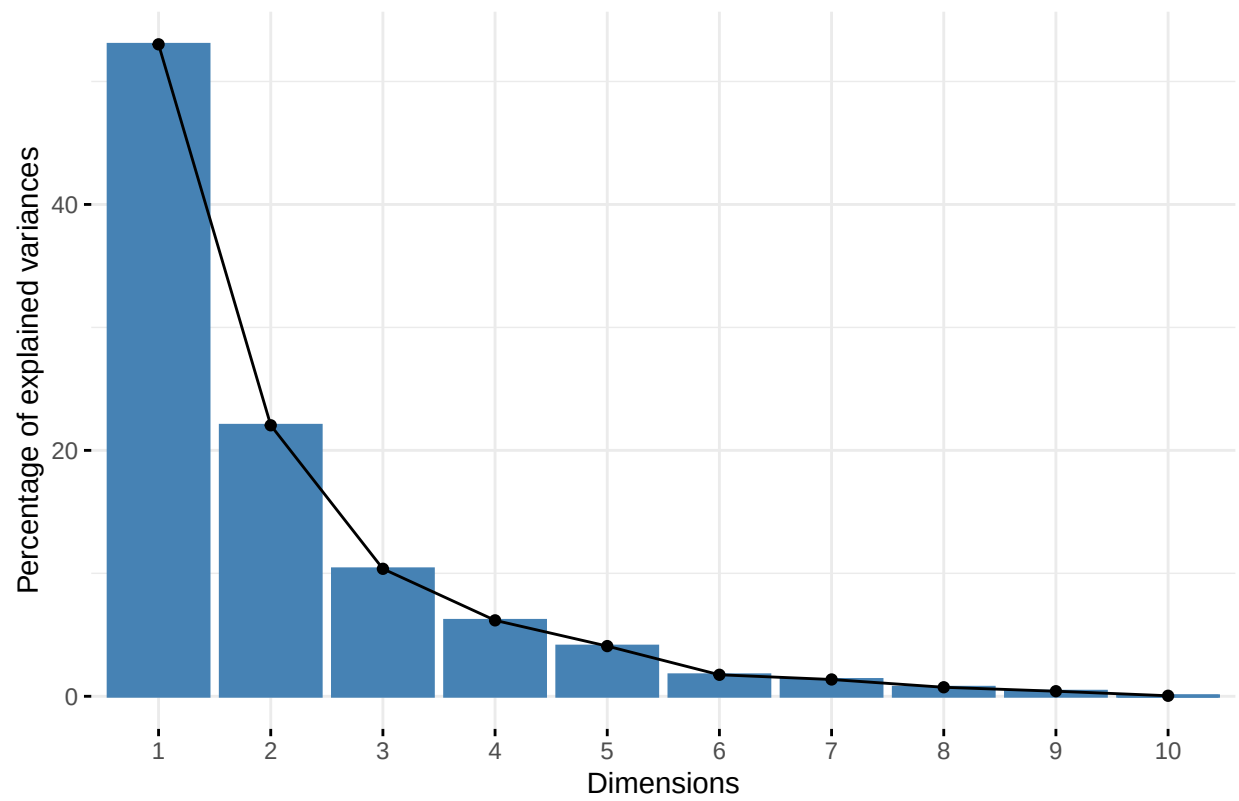
```
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.294"
```

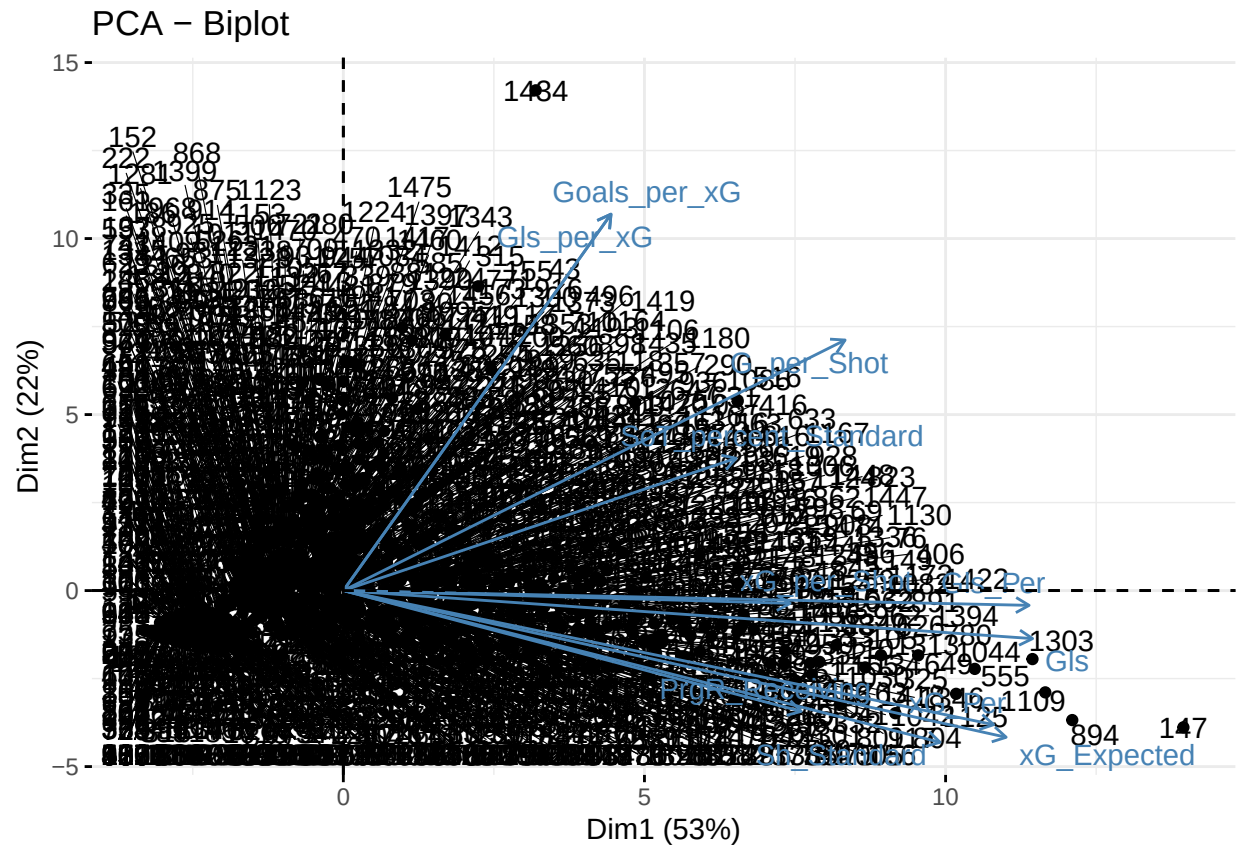
```
# Plot the variance explained
```

```
fviz_eig(outfield_pca)
```

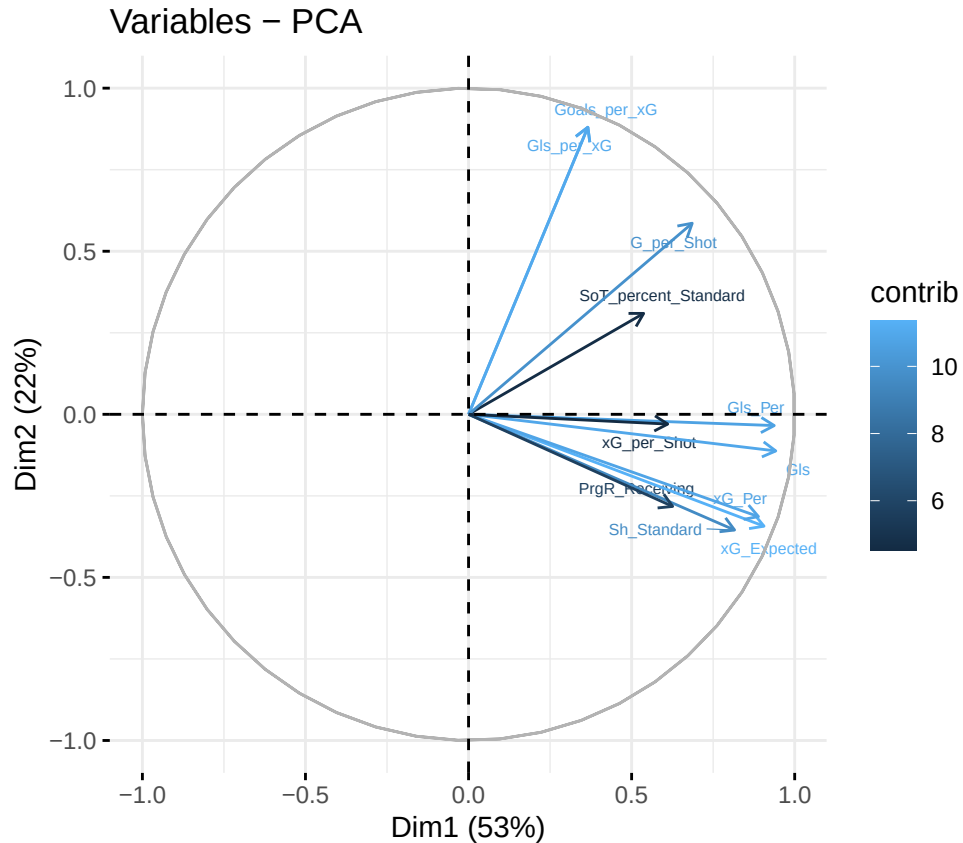
Scree plot



```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```



```
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
  col.var = "contrib",
  labelsize = 2, repel = TRUE)
```



```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

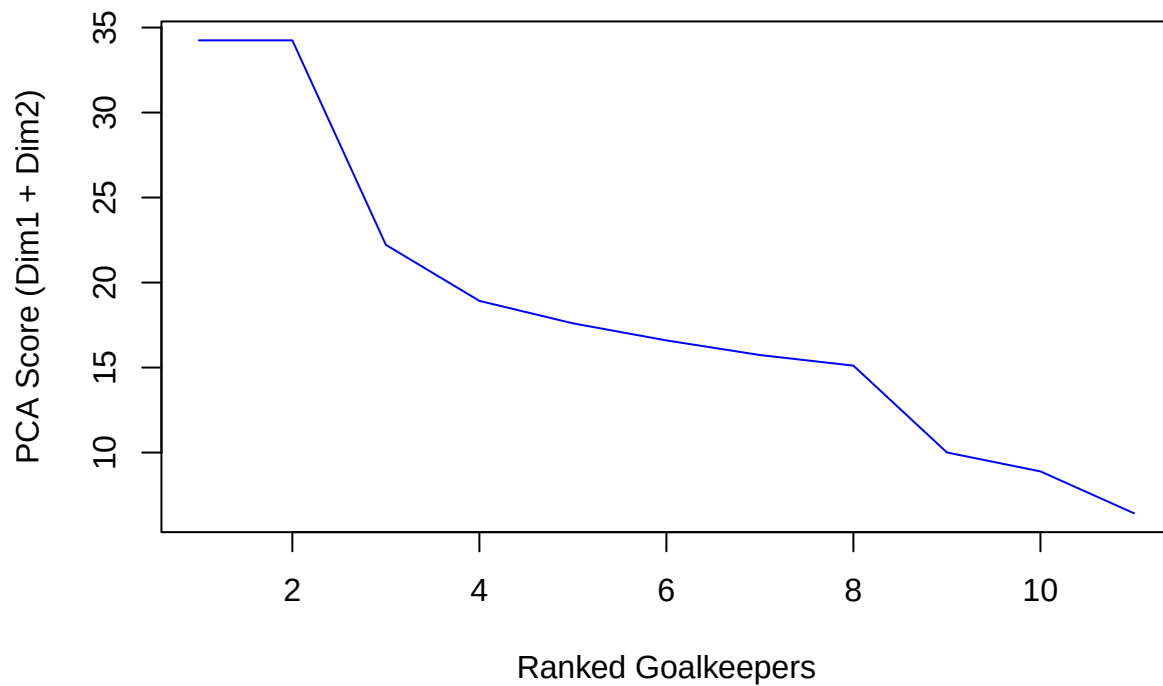
# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 20.57007
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```

PCA Score Distribution



```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##      j1 j2 chi
## ESE  4  8   6
## EDE  4  2 NaN
```

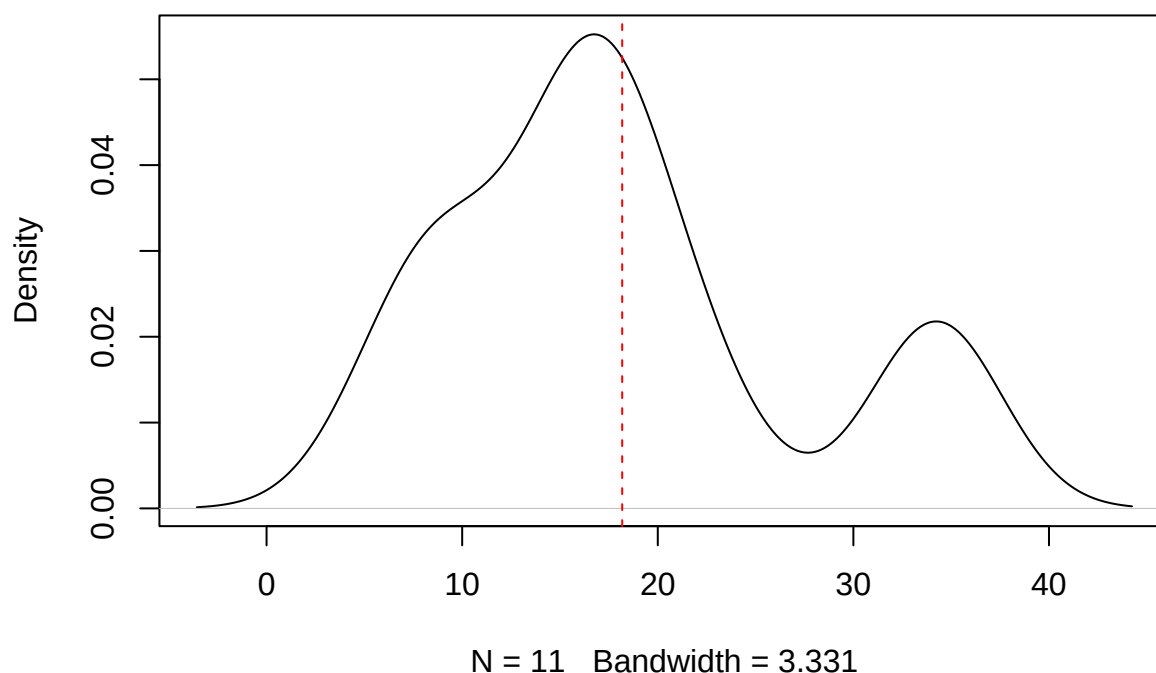
```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 27.36117
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```

Density of PCA Scores



#####

#> GOALSCORER RATING

#Verify if the goals are per 90 or per shot - think they might be per 90

```
outfield_DPR_OCR_gsr <- outfield_gsr_data %>% filter(Min_Playing.Time > 900) %>% mutate(GoalScorer_Rtg =
  (ifelse(xG_Expected > 0, Gls / xG_Expected, 0) * variable_contrib["Gls_per_xG", "Weight"]) +
  (SoT_percent_Standard * variable_contrib["SoT_percent_Standard", "Weight"]) +
  (ifelse(Sh_Standard > 0, Gls / Sh_Standard, 0) * variable_contrib["G_per_Shot", "Weight"]) +
  (xG_per_Shot * variable_contrib["xG_per_Shot", "Weight"]) +
  (PrgR_Receiving * variable_contrib["PrgR_Receiving", "Weight"])
) / Min_Playing.Time) %>% arrange(desc(GoalScorer_Rtg))
```

```
outfield_gsr_awards <- outfield_DPR_OCR_gsr %>% filter(Min_Playing.Time >= 2160)
outfield_GSR <- outfield_DPR_OCR_gsr
```

#Correlation work between GA and GKR

```
cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$Gls_Per, use = "complete.obs", method = "spearman")
```

Sanity check GSR with xG

```
## [1] 0.647379
```

```
cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$xG_Per, use = "complete.obs", method = "spearman")
```

```
## [1] 0.6600422
```

```
summary_model_GSR <- lm(Gls_Per ~ GoalScorer_Rtg, data = outfield_GSR)
summary(summary_model_GSR)$r.squared
```

```
## [1] 0.3614604
```

Reassigning a few table names

```
outfield_OCR <- outfield_DPR_OCR
```

Overall Player Ratings

```
outfield_DPR_filtered <- outfield_DPR %>% select(Squad, Comp, Player, Pos, Poss.Adj.DPR, Min_Playing.Time)
outfield_OCR_filtered <- outfield_OCR %>% select(Squad, Comp, Player, Pos, Poss.adj.OCR, Min_Playing.Time)
outfield_GSR_filtered <- outfield_GSR %>% select(Squad, Comp, Player, Pos, GoalScorer_Rtg, Min_Playing.Time)
```

```
player_master <- full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player") %>% full_join(outfield_GSR_filtered, by = "Player")
```

Take the four ratings dataframes and join them before doing clustering on them

```
## Warning in full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player"): Detected an unexpected many-to-many relationship
## i Row 13 of 'x' matches multiple rows in 'y'.
## i Row 84 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
## Warning in full_join(., outfield_GSR_filtered, by = "Player"): Detected an unexpected many-to-many relationship
## i Row 13 of 'x' matches multiple rows in 'y'.
## i Row 80 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
player_master <- player_master %>%
  mutate(
    DPR_std = (Poss.Adj.DPR - min(Poss.Adj.DPR, na.rm=TRUE)) / (max(Poss.Adj.DPR, na.rm=TRUE) - min(Poss.Adj.DPR, na.rm=TRUE)),
    OCR_std = (Poss.adj.OCR - min(Poss.adj.OCR, na.rm=TRUE)) / (max(Poss.adj.OCR, na.rm=TRUE) - min(Poss.adj.OCR, na.rm=TRUE)),
    GSR_std = (GoalScorer_Rtg - min(GoalScorer_Rtg, na.rm=TRUE)) / (max(GoalScorer_Rtg, na.rm=TRUE) - min(GoalScorer_Rtg, na.rm=TRUE))
  )
```

```
player_master <- player_master %>% mutate(
  Squad= coalesce(Squad, Squad.x, Squad.y),
  Comp= coalesce(Comp, Comp.x, Comp.y),
  Pos = coalesce(Pos.x, Pos.y, Pos),
  Min_Playing_Time = coalesce(Min_Playing.Time.x, Min_Playing.Time.y, Min_Playing.Time))
```

```
player_master_final <- player_master %>% select(Player, Squad, Comp, Pos, Poss.Adj.DPR, Poss.adj.OCR, GSR)
```

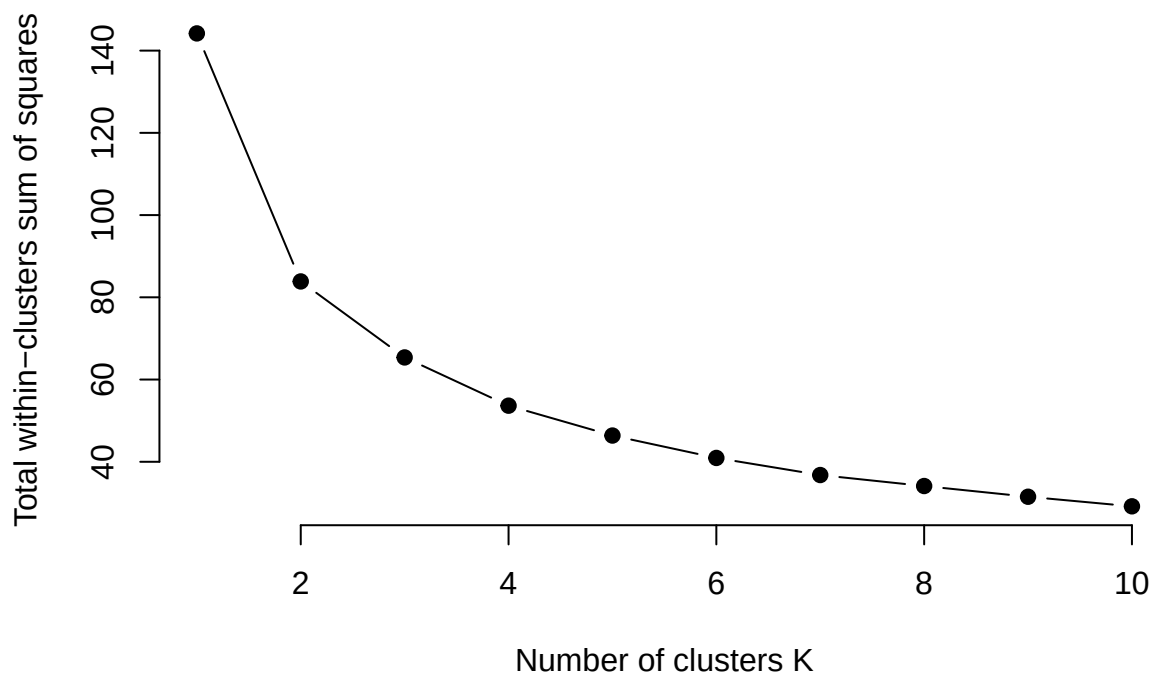
Cluster the players to look for any natural clusterings then run PCA on each cluster

```
player_clustering <- player_master_final %>%
  select(DPR_std, OCR_std, GSR_std) %>%
  mutate(across(everything(), ~ ifelse(is.na(.), 0, .)))

wss <- function(k) {
  kmeans(player_clustering, k, nstart = 25)$tot.withinss
}

k.values <- 1:10
wss_values <- map_dbl(k.values, wss)

plot(k.values, wss_values, type = "b",
     pch = 19, frame = FALSE,
     xlab = "Number of clusters K",
     ylab = "Total within-clusters sum of squares")
```



```
library(cluster)

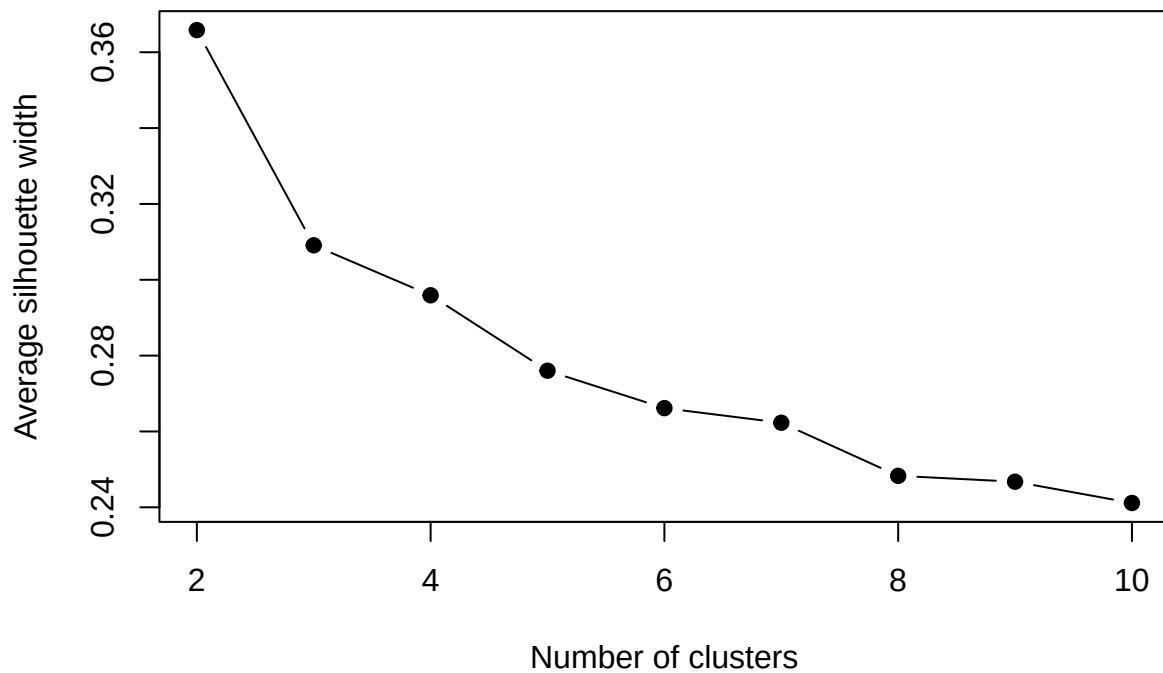
sil_width <- function(k){
  pam(player_clustering, k)$silinfo$avg.width
}
```

```

k.values <- 2:10
sil_values <- map_dbl(k.values, sil_width)

plot(k.values, sil_values, type = "b", pch = 19,
     xlab = "Number of clusters",
     ylab = "Average silhouette width")

```



```

set.seed(212)
kmeans_result <- kmeans(player_clustering, centers = 2, nstart = 25)

player_master_final$Cluster <- kmeans_result$cluster

```

```

run_pca_weights <- function(data_numeric){
  # Perform PCA
  pca <- PCA(data_numeric, graph = FALSE)

  # Extract eigenvalues (variance explained by each component)
  eig <- pca$eig

  # Extract contributions of variables to PC1 and PC2
  var_contrib <- data.frame(pca$var$contrib)

  # Sum contribution of Dim.1 and Dim.2 and scale them to sum to 1
  var_contrib <- var_contrib %>%
    mutate(Total_Contrib = Dim.1 + Dim.2,

```

```

    Weight = Total_Contrib / sum(Total_Contrib))

  # Return PCA object, weights, and eigenvalues
  return(list(pca = pca, weights = var_contrib, eig = eig))
}

player_clustering_final <- player_clustering
player_clustering_final$Player <- player_master_final$Player
player_clustering_final$Squad <- player_master_final$Squad
player_clustering_final$Comp <- player_master_final$Comp
player_clustering_final$Pos <- player_master_final$Pos
player_clustering_final$Cluster <- player_master_final$Cluster
player_clustering_final$Min_Playing_Time <- player_master_final$Min_Playing_Time

clustered_pca_results <- player_clustering_final %>%
  group_split(Cluster) %>%
  map(~run_pca_weights(.x %>% select(DPR_std, OCR_std, GSR_std)))

cluster_weights <- map(clustered_pca_results, ~.x$weights %>% select(Weight) %>% rownames_to_column("Me
names(cluster_weights) <- paste0("Cluster_", 1:length(cluster_weights))

player_clusters <- player_clustering_final %>% group_split(Cluster)

cluster_ratings <- map2(
  player_clusters,
  cluster_weights,
  ~ .x %>% mutate(
    Overall_Rating =
      DPR_std * .y$Weight[.y$Metric == "DPR_std"] +
      OCR_std * .y$Weight[.y$Metric == "OCR_std"] +
      GSR_std * .y$Weight[.y$Metric == "GSR_std"]
  )
)

player_master_final <- bind_rows(cluster_ratings)
player_master_final <- player_master_final %>% arrange(desc(Overall_Rating))

pl_ratings_final <- player_master_final %>% filter(Comp == "Premier League")
laliga_ratings_final <- player_master_final %>% filter(Comp == "La Liga")
ligue1_ratings_final <- player_master_final %>% filter(Comp == "Ligue 1")
bundesliga_ratings_final <- player_master_final %>% filter(Comp == "Bundesliga")
seriea_ratings_final <- player_master_final %>% filter(Comp == "Serie A")

```

Finally decide the awards and the team of the year based on minutes - 2260 minutes played (24 games)

```

player_master_awards <- player_master_final %>% filter(Min_Playing_Time >= 2260)
pl_ratings_awards <- pl_ratings_final %>% filter(Min_Playing_Time >= 2260)
laliga_ratings_awards <- laliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
ligue1_ratings_awards <- ligue1_ratings_final %>% filter(Min_Playing_Time >= 2260)

```

```
seriea_ratings_awards <- seriea_ratings_final %>% filter(Min_Playing_Time >= 2260)
bundesliga_ratings_awards <- bundesliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
```

Overall Team Ratings

```
team_ratings_weighted <- player_master_final %>%
  group_by(Squad, Comp) %>%
  summarise(
    Weighted_Team_Rating = sum(Overall_Rating * Min_Playing_Time, na.rm = TRUE) / sum(Min_Playing_Time,
    Total_Minutes = sum(Min_Playing_Time, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Weighted_Team_Rating))

pl_team_ratings <- team_ratings_weighted %>% filter(Comp == "Premier League")
laliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "La Liga")
ligue1_team_ratings <- team_ratings_weighted %>% filter(Comp == "Ligue 1")
bundesliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "Bundesliga")
seriea_team_ratings <- team_ratings_weighted %>% filter(Comp == "Serie A")
```

No Keeper Ratings

```
pl_ratings_table <- pl_team_ratings %>% select(Squad, Weighted_Team_Rating)
pl_table <- prem_league_table %>% select(Squad) %>% rownames_to_column()
colnames(pl_table)[1] <- "Position"
pl_xg_table <- prem_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

pl_combined_table <- pl_ratings_table %>%
  left_join(pl_table, by = c("Squad")) %>%
  left_join(pl_xg_table, by = c("Squad"))

pl_combined_table$Position <- as.numeric(pl_combined_table$Position)

cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Position, method = "spearman")
```

Premier League

```
## [1] -0.8135338
```

```
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Pts, method = "spearman")
```

```
## [1] 0.8194133
```

```
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$xGD, method = "spearman")
```

```
## [1] 0.7002633
```

```

ggplot(pl_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(pl_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

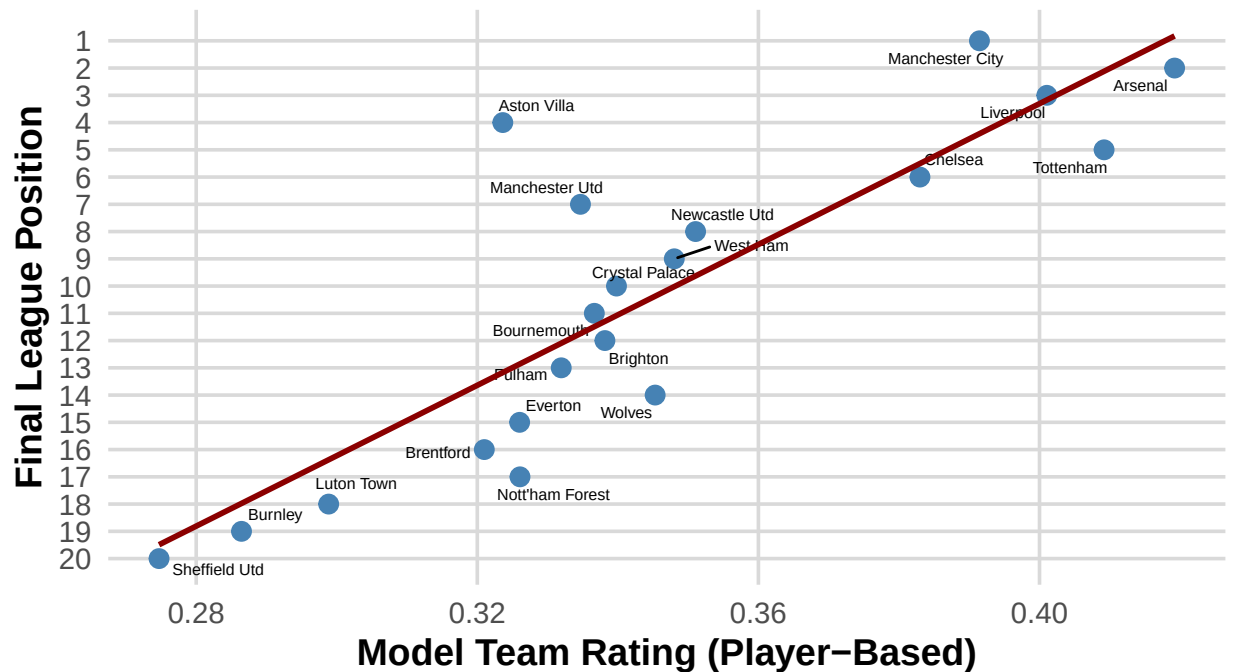
```

## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?

```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
laliga_ratings_table <- laliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
laliga_table <- laliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(laliga_table)[1] <- "Position"
laliga_xg_table <- laliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

laliga_combined_table <- laliga_ratings_table %>%
  left_join(laliga_table, by = c("Squad")) %>%
  left_join(laliga_xg_table, by = c("Squad"))

laliga_combined_table$Position <- as.numeric(laliga_combined_table$Position)

cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Position, method = "spearman")
```

La Liga

```
## [1] -0.7308271
```

```
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.7188563
```

```
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.593985
```

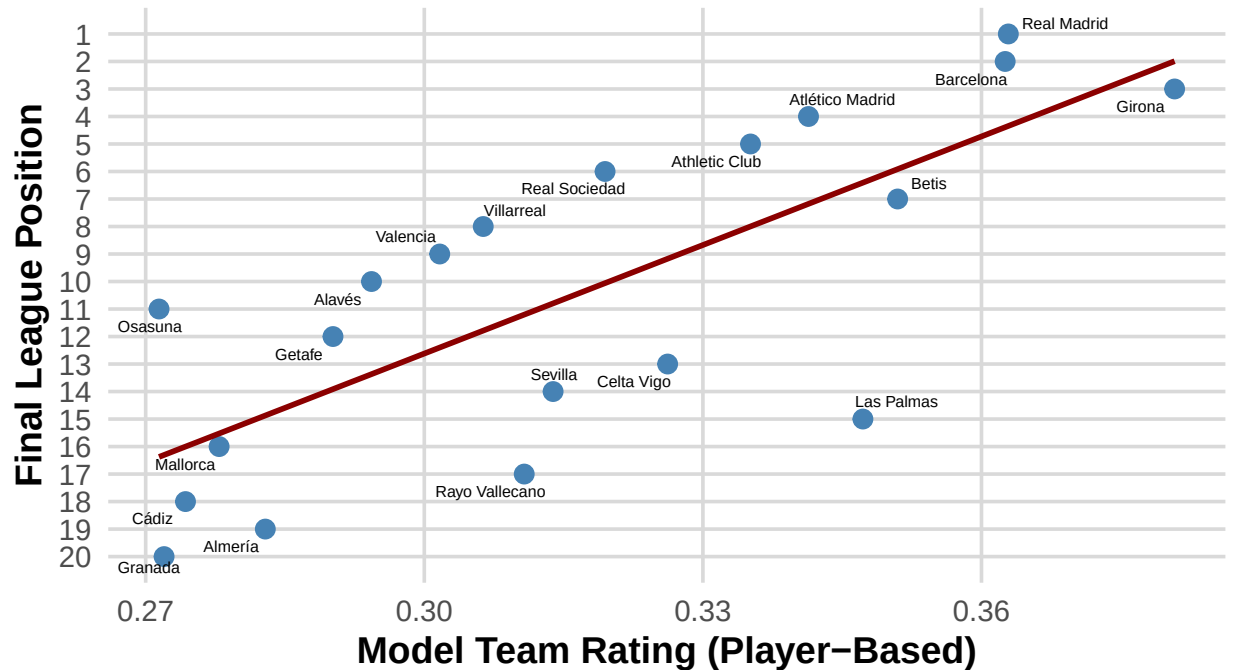
```
ggplot(laliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(laliga_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



Line: Linear regression fit | Labels: Squads

Ligue 1

```

ligue1_ratings_table <- ligue1_team_ratings %>% select(Squad, Weighted_Team_Rating)
ligue1_table <- ligue1_league_table %>% select(Squad) %>% rownames_to_column()
colnames(ligue1_table)[1] <- "Position"
ligue1_xg_table <- ligue1_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

ligue1_combined_table <- ligue1_ratings_table %>%
  left_join(ligue1_table, by = c("Squad")) %>%
  left_join(ligue1_xg_table, by = c("Squad"))

ligue1_combined_table$Position <- as.numeric(ligue1_combined_table$Position)

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Position, method = "spearman")

## [1] -0.8452012

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Pts, method = "spearman")

## [1] 0.8487353

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$xGD, method = "spearman")

## [1] 0.7172343

```

```

ggplot(ligue1_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(ligue1_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

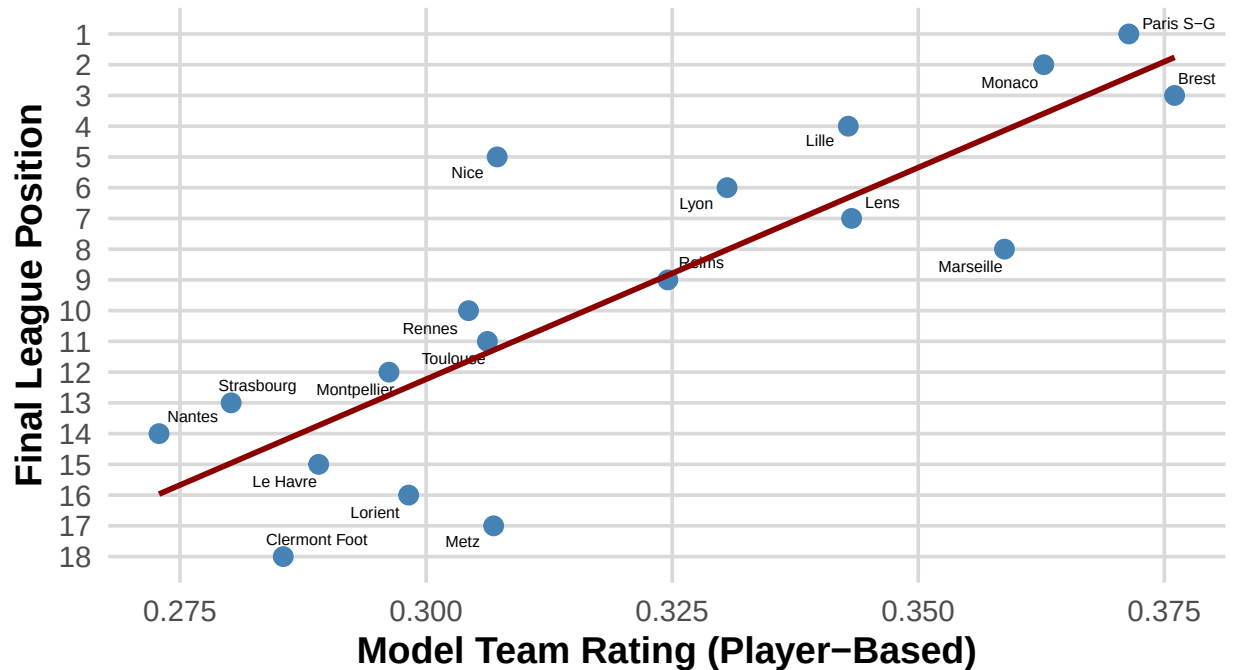
```

## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a 'group' aesthetic or to convert a numerical
##   variable into a factor?

```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
bundesliga_ratings_table <- bundesliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
bundesliga_table <- bundesliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(bundesliga_table)[1] <- "Position"
bundesliga_xg_table <- bundesliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

bundesliga_combined_table <- bundesliga_ratings_table %>%
  left_join(bundesliga_table, by = c("Squad")) %>%
  left_join(bundesliga_xg_table, by = c("Squad"))

bundesliga_combined_table$Position <- as.numeric(bundesliga_combined_table$Position)

cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Position, method = "spearman")
```

Bundesliga

```
## [1] -0.748194
```

```
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.7542705
```

```
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.6594427
```

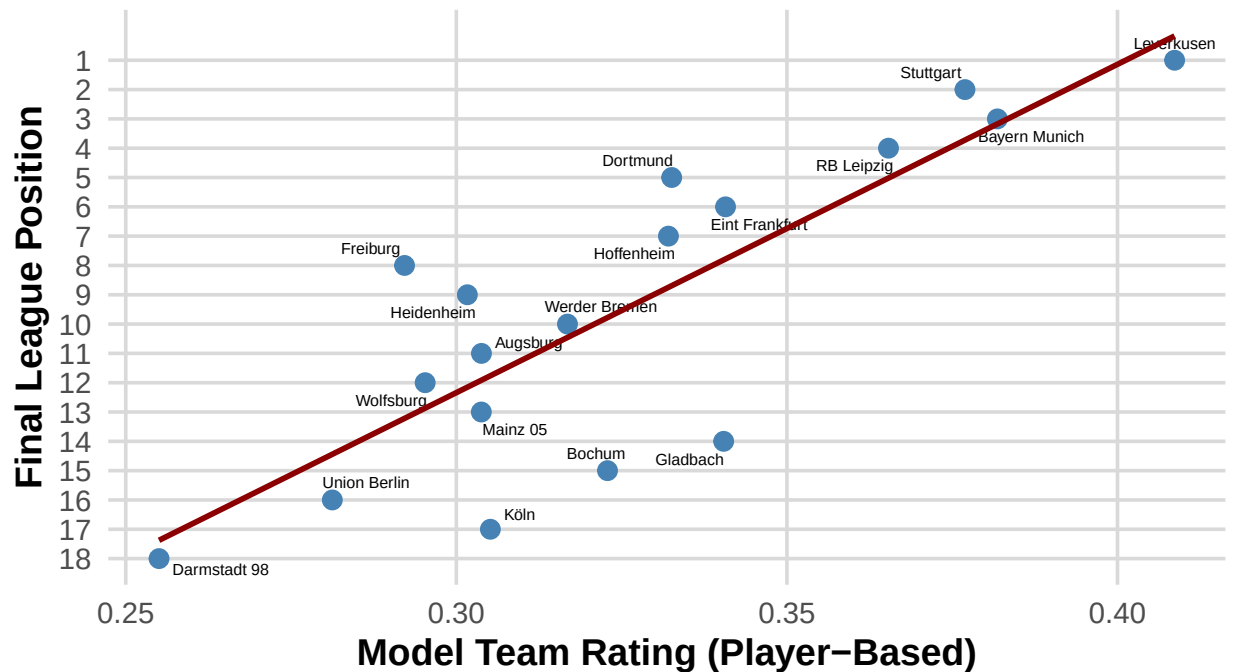
```
ggplot(bundesliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(bundesliga_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



```
seriea_ratings_table <- seriea_team_ratings %>% select(Squad, Weighted_Team_Rating)
seriea_table <- seriea_league_table %>% select(Squad) %>% rownames_to_column()
colnames(seriea_table)[1] <- "Position"
seriea_xg_table <- seriea_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

seriea_combined_table <- seriea_ratings_table %>%
  left_join(seriea_table, by = c("Squad")) %>%
  left_join(seriea_xg_table, by = c("Squad"))

seriea_combined_table$Position <- as.numeric(seriea_combined_table$Position)

cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Position, method = "spearman")
```

Serie A

```
## [1] -0.6947368
```

```
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Pts, method = "spearman")
```

```
## [1] 0.6894999
```

```
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8300752
```

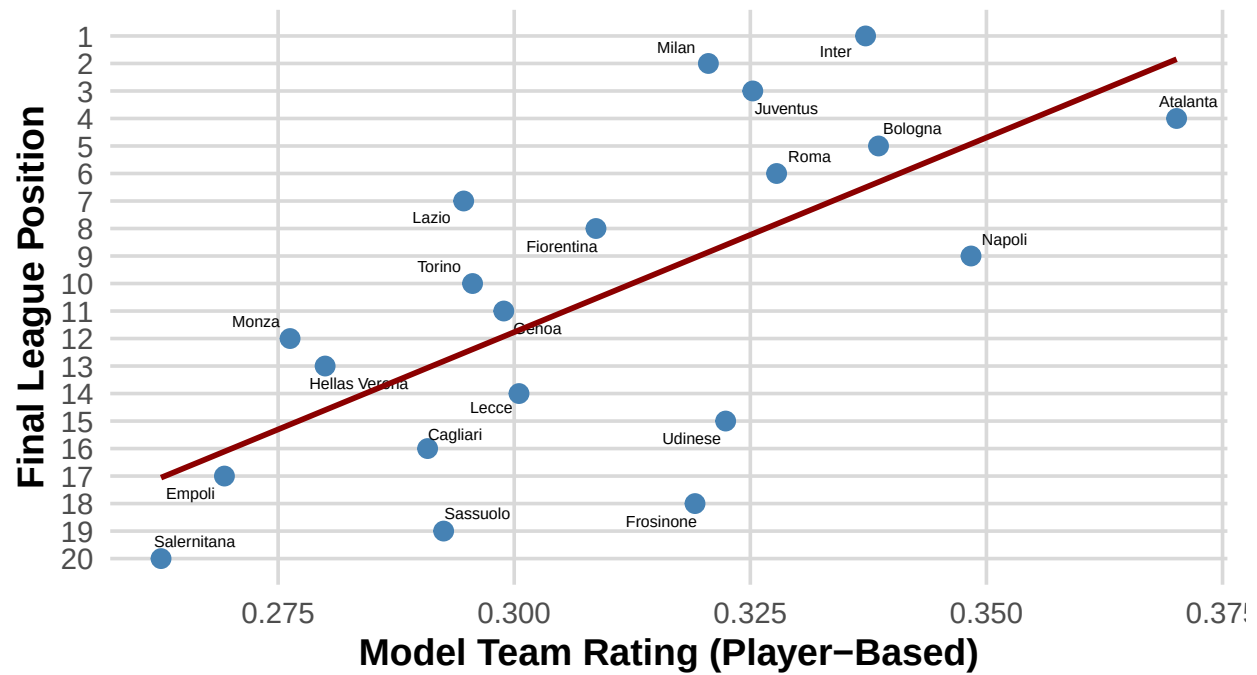
```
ggplot(seriea_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +  
  geom_point(size = 3, color = "steelblue") +  
  geom_text_repel(size = 2, max.overlaps = 20) +  
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +  
  scale_y_reverse(breaks = 1:max(seriea_combined_table$Position, na.rm = TRUE)) +  
  labs(  
    title = "Team Model Rating vs Actual League Position",  
    subtitle = "Higher team rating should correlate with higher league finish",  
    x = "Model Team Rating (Player-Based)",  
    y = "Final League Position",  
    caption = "Line: Linear regression fit | Labels: Squads"  
  ) +  
  theme_minimal(base_size = 14) +  
  theme(  
    plot.title = element_text(face = "bold", size = 18),  
    plot.subtitle = element_text(size = 14, color = "gray40"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.major = element_line(color = "gray85"),  
    panel.grid.minor = element_blank()  
  )
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.  
## i This can happen when ggplot fails to infer the correct grouping structure in  
##   the data.  
## i Did you forget to specify a 'group' aesthetic or to convert a numerical  
##   variable into a factor?
```

Team Model Rating vs Actual League Position

Higher team rating should correlate with higher league finish



Line: Linear regression fit | Labels: Squads